ML MENTOR LAB

# Code Analysis & Command Guide

A source-level walkthrough of the Mentor Lab education platform: setup commands, backend controllers and routes, frontend pages, data models, request flows, security findings, debugging playbook, and viva-ready explanations.

React + Vite | Node.js + Express 5 | MongoDB + Mongoose | JWT + bcrypt | GridFS | Razorpay / UPI | Groq AI

> Use this guide as a reading companion to the source code. Every important function is explained in context, with file references so you can jump directly to the implementation. Secrets and real credentials are deliberately excluded.


**Mentor Lab**
Full-stack education website project

**Edition 1.0**
Code analysis, commands and viva preparation

**2026**
BCA / BBA / Biotechnology learning platform

# Contents


**PART I**

## Orientation

1. Project at a Glance
2. Command Center
3. Architecture and Request Lifecycle

**PART II**

## Backend Code Analysis

4. Server Bootstrap and MongoDB
5. Authentication, JWT and OTP
6. Users, Profiles and Activity
7. Admin APIs and Dashboard Data
8. Notes, PDF Extraction and GridFS
9. Previous Year Questions
10. Mock Tests: Upload and PDF Parsing
11. Mock Tests: Access, Payment and Scoring
12. Certificates and Completion
13. Payments, AI and TTS

**PART III**

## Frontend Code Analysis

14. React App, Routes and Guards
15. Authentication Screens
16. Mock Test Interface
17. Note Viewer and Markdown Parser
18. Analytics, Profile and Certificates
19. Admin Dashboard UI
20. PYQ, Lectures, Chatbot and Compiler

**PART IV**

## Cross-Cutting Analysis

21. Data Model Atlas
22. API Reference
23. End-to-End Flows
24. Security and Code Quality Audit
25. Debugging Playbook

**PART V**

## Viva and Revision

26. Numbers and Details to Memorize
27. Rapid-Fire "Why" Answers
28. File Map and Final Checklist

**HOW TO USE**

## Reading Paths

New to the project: Chapters 1, 3, 21, 23

Preparing for viva: Chapters 5, 7, 10, 11, 12, 26, 27

Debugging a failure: Chapters 2, 24, 25

Explaining the frontend: Chapters 14, 16, 17, 18


**Scope and accuracy.** This guide describes the code that is present in the project folder. Where the implementation is incomplete, insecure, duplicated, or inconsistent, that is stated directly rather than hidden. The guide never prints the values of `MONGO_URI`, `JWT_SECRET`, `GROQ_API_KEY`, SMTP passwords, Razorpay secrets, or the hard-coded admin credentials. Commands use placeholder values and are safe to adapt.

I

# Orientation

What the project is, how it is organised, which commands actually run it, and how a browser request travels through the system. Read this part before the detailed code walkthrough.

CHAPTER 01

# Project at a Glance

> Mentor Lab is a full-stack study platform for BCA, BBA and Biotechnology students. It combines course material, previous-year papers, mock tests, performance analytics and certificates in one React application backed by an Express API and MongoDB.

## 1.1 What the platform does

The application is not a single-purpose website. It has several independent learning modules that share one authentication system and one database:

| Module | Student-facing purpose | Primary code |
|---|---|---|
| Course discovery | Landing pages, course categories, subject lists and roadmaps for BCA, BBA and Biotech. | `frontend/src/pages/bca.jsx`, `bba.jsx`, `biotech.jsx`, category pages |
| Study notes | Browse notes, read extracted PDF text in a web viewer, download the original PDF, and track completion. | `pages/StudyNotes.jsx`, `pages/NoteViewer.jsx`, `routes/notesRoutes.js` |
| Previous year questions | Choose a university and subject, then open or download a PYQ PDF. | `pages/PYQ.jsx`, `routes/pyqRoutes.js` |
| Mock tests | Take timed MCQ tests, use a CBT-style question palette, pay for locked tests, submit answers and review results. | `pages/MockTest.jsx`, `pages/TestReview.jsx`, `routes/mockTestRoutes.js` |
| Analytics | See tests taken, accuracy, score trend, subject performance, monthly activity and a day streak. | `pages/PerformanceAnalytics.jsx` |
| Certificates | Earn a course certificate after reading every note and attempting every test in that course. | `routes/certificateRoutes.js`, `pages/CertificatePage.jsx` |
| Admin console | View stats, upload notes/PYQs/mock tests, manage purchases, unlock tests and set payment details. | `pages/AdminDashboard.jsx`, `routes/adminRoutes.js` |
| AI assistant | Ask study questions in English or Hindi, listen to answers, and request note explanations. | `components/Chatbot.jsx`, `controllers/aiController.js` |
| Code compiler | Run C, C++, Java, Python and JavaScript through a public compiler API; preview HTML/CSS/JS locally. | `components/CodeCompiler.jsx` |

## 1.2 Technology stack

**Frontend**

React 19, React Router 7, Vite 8, Axios, Recharts, Firebase SDK (partially configured), custom CSS.

**Backend**

Node.js, Express 5, Mongoose 9, JWT, bcryptjs, Multer, Nodemailer, pdf-parse, Razorpay SDK.

**Data and services**

MongoDB Atlas or local MongoDB, GridFS buckets `pdfs` and `pyqs`, Groq chat API, Google Translate TTS, Wandbox compiler API.

## 1.3 Repository layout

```
Monitor Lab/
├── backend/
│   ├── src/index.js            Express entry point
│   ├── src/db.js               MongoDB connection
│   ├── routes/                 8 route modules
│   ├── controllers/            auth, AI, TTS
│   ├── middleware/             JWT protect middleware
│   ├── model/                  10 Mongoose models
│   ├── utils/                  token + mailer helpers
│   ├── createAdmin.js           admin bootstrap script
│   ├── seedMockTest.js          large mock-test seed script
│   ├── uploads/                 disk files (QR images, legacy PDFs)
```

```
│   └── package.json
├── frontend/
│   ├── src/App.jsx              route table
│   ├── src/pages/               22 page components
│   ├── src/components/          guards, chatbot, compiler, modal
│   ├── src/utils/markdown.js    PDF-text-to-HTML parser
│   ├── src/data/                course category data
│   ├── src/css_files/           page-specific styles
│   └── package.json
└── .gitignore
```

**NAMING INCONSISTENCY**

The public brand is **Mentor Lab**, but the frontend package is named `drona-gyan` and some UI text still says "Drona Gyan" or "Monitor Lab". The backend package is named `monitor-lab-backend`. This is a presentation inconsistency, not a runtime failure, but it should be standardised before a formal demo.

## 1.4 The central design idea

Every student action is tied to the same identity: the MongoDB `User` document referenced in the JWT. That one identity links purchases, test results, read notes and certificates:

**ONE IDENTITY, MANY MODULES**

**JWT** `{ id }` signed for 7 days → **protect** loads `User` and removes password → **Feature route** uses `req.user._id` → **MongoDB** Purchase, Result, readNotes, Certificate

This is why the project can be explained simply in a viva: authentication establishes `req.user`; every protected feature then uses that user id to read or write its own collection.

## 1.5 Strengths worth highlighting

- **Complete feature loop:** content → practice → scoring → progress → certificate.
- **Separation of concerns:** routes, controllers, models, utilities and middleware are separated on the backend.
- **Real file storage:** PDFs are stored in MongoDB GridFS, not just as paths in the database.
- **Payment fallback:** Razorpay when configured, manual UPI confirmation otherwise.
- **Security awareness in auth:** bcrypt hashing, hashed OTPs, OTP expiry, resend cooldown, attempt limits and one-time reset tokens.
- **Resilient note reading:** existing PDFs can be lazily converted to text on first view.

## 1.6 Weaknesses to acknowledge honestly

| Weakness | Why it matters |
|---|---|
| Some admin/data endpoints are public | `/api/admin/stats`, `/students`, `/courses` and `POST /courses` do not use `protect` or an admin check. |
| Upload/delete routes trust any logged-in user | Note and PYQ upload/delete routes use only `protect`, not `req.user.isAdmin`. |
| Manual UPI flow is not verified | A student-supplied UTR immediately unlocks a paid test when Razorpay is not configured. |
| Frontend route guards are cosmetic | `ProtectedRoute` checks only that a token string exists; it does not validate expiry or signature. |
| Hard-coded localhost URLs | Chatbot and TTS calls use `http://localhost:5000`, so they break when deployed. |
| Heuristic PDF parser | Mock-test and note parsing depend on text patterns, so unusual PDFs can produce imperfect questions or headings. |
| Inconsistent stats logic | `performanceStats.avgScore` and `hoursLearnt` are not recalculated, and rank stays `"N/A"`. |
| Duplicate course model usage | The `Course` collection is used for counts, while the actual UI course lists come from static frontend data files. |

**VIVA FRAMING**

Do not claim the project is production-perfect. Say: “The core learning loop is complete and functional; the next engineering pass would harden authorisation, verify manual payments, centralise API URLs, and add automated tests.” That answer is stronger than pretending there are no gaps.

CHAPTER 02

# Command Center

> The exact commands needed to install, run, build, inspect, seed and troubleshoot the project on Windows. The project uses npm, not yarn or pnpm.

## 2.1 Prerequisites

| Tool | Purpose | Check command |
|---|---|---|
| Node.js 20+ | Runs Vite and the Express server. This machine has Node 24. | `node -v` |
| npm | Installs dependencies and runs scripts. | `npm.cmd -v` |
| MongoDB | Database. Can be local or Atlas. | `mongosh --version` |
| Git | Optional version control. | `git --version` |

**WINDOWS POWERSHELL NOTE**

If PowerShell blocks `npm.ps1` with an execution-policy error, use `npm.cmd` instead of `npm`. This is the reason every npm example below uses `npm.cmd`. Always quote paths that contain spaces.

## 2.2 Install dependencies

**Backend dependencies**

Installs Express, Mongoose, JWT, bcrypt, Multer, Nodemailer, pdf-parse and Razorpay.

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\backend"
npm.cmd install
```

**Frontend dependencies**

Installs React, React Router, Axios, Vite, ESLint, Recharts and Firebase.

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\frontend"
npm.cmd install
```

## 2.3 Environment variables

The backend reads a `.env` file from the `backend` directory. The current file declares these keys; values are intentionally not reproduced here:

| Key | Used by | Effect |
|---|---|---|
| `PORT` | `src/index.js` | HTTP port, default 5000. |
| `MONGO_URI` | `src/db.js` | MongoDB connection string. If missing, startup throws. |
| `JWT_SECRET` | `generateToken.js`, `authMiddleware.js`, reset flow | Signs and verifies login and password-reset tokens. |
| `GROQ_API_KEY` | `controllers/aiController.js` | Authenticates AI chat and note-explanation requests. |
| `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM` | `utils/mailer.js` | Optional; without them the OTP email is skipped. |
| `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET` | `routes/mockTestRoutes.js` | Optional; enables real Razorpay orders and signature verification. |
| `NODE_ENV` | forgot-password response | In non-production, an unsent OTP is returned as `devOtp`. |

**NEVER COMMIT THE .ENV FILE**

Treat `MONGO_URI`, `JWT_SECRET`, `GROQ_API_KEY`, SMTP credentials and Razorpay secrets as private. Rotate any value that has been committed or shared. The guide and generated documents do not contain their values.

## 2.4 Run the development servers

Use two terminals. The backend must be running before the frontend makes API calls.

**Terminal 1 — backend with nodemon**

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\backend"
npm.cmd run dev
```

Expected output: `Connecting to MongoDB...`, `MongoDB Connected: ...`, then `Server running on port 5000`.

**Terminal 2 — frontend with Vite**

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\frontend"
npm.cmd run dev
```

Vite prints a local URL, usually `http://localhost:5173`. The dev proxy forwards `/api` and `/uploads` to port 5000.

## 2.5 Production-style run

The Express server serves `frontend/dist` if it exists. Build the frontend first, then start the backend:

**Build and serve from one origin**

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\frontend"
npm.cmd run build

cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\backend"
npm.cmd start
```

Open `http://localhost:5000`. React Router deep links work because the backend has an SPA catch-all route that returns `index.html`.

## 2.6 Script reference

| Directory | Command | What it does |
|---|---|---|
| backend | `npm.cmd start` | Runs `node src/index.js`. No auto-reload. |
| backend | `npm.cmd run dev` | Runs `nodemon src/index.js`. Restarts on file changes. |
| frontend | `npm.cmd run dev` | Starts the Vite development server with HMR and API proxy. |
| frontend | `npm.cmd run build` | Creates an optimized production bundle in `frontend/dist`. |
| frontend | `npm.cmd run preview` | Serves the built bundle locally for a production-like check. |
| frontend | `npm.cmd run lint` | Runs ESLint over the frontend source. |
| backend | `node createAdmin.js` | Creates or promotes an admin. The script currently contains hard-coded credentials and should be changed first. |
| backend | `node seedMockTest.js` | Inserts a large seeded mock test. Review it before running against a real database. |

**ADMIN BOOTSTRAP WARNING**

`backend/createAdmin.js` contains plain-text admin credentials in source code. Do not run it unchanged on a shared database, do not share the file, and do not copy its credentials into notes or slides. The correct fix is to read the email and password from environment variables or a one-time prompt, then remove the defaults.

## 2.7 Database commands

### Open the database shell

```
mongosh "$env:MONGO_URI"
```

If the connection string is not in the current shell, paste it interactively rather than storing it in shell history.

### Useful mongosh inspection commands

```
show dbs
use mentor-lab
show collections
db.users.countDocuments()
db.users.find({}, { name: 1, email: 1, isAdmin: 1 }).limit(5)
db.mocktests.countDocuments()
db.results.countDocuments()
db.purchases.find().sort({ createdAt: -1 }).limit(5)
db.pdfs.files.find().limit(5)
db.pyqs.files.find().limit(5)
```

### Backup and restore

```
mongodump --uri="$env:MONGO_URI" --out="C:\backup\mentorlab"
mongorestore --uri="$env:MONGO_URI" "C:\backup\mentorlab"
```

GridFS contents are included when the database is dumped, because they live in the `pdfs` and `pyqs` collections.

## 2.8 API smoke tests

Use fake data in examples. Replace placeholders locally; never paste real credentials into screenshots or reports.

### Health-style checks with curl.exe

```
curl.exe -i http://localhost:5000/api/notes
curl.exe -i http://localhost:5000/api/pyq
curl.exe -i http://localhost:5000/api/admin/stats
curl.exe -i http://localhost:5000/api/payment/settings
```

These endpoints are intentionally public in the current code. A `200` response confirms the server and database are reachable.

### Login and authenticated request

```
$body = @{ email = "student@example.com"; password = "YourTestPassword1!" } | ConvertTo-Json
$login = Invoke-RestMethod -Method Post -Uri "http://localhost:5000/api/auth/login" -ContentType
"application/json" -Body $body
$headers = @{ Authorization = "Bearer $($login.token)" }
Invoke-RestMethod -Uri "http://localhost:5000/api/auth/profile" -Headers $headers
```

The same token is used by `protect` for results, certificates, purchases and profile routes.

## 2.9 Port and process troubleshooting

**Find what is using port 5000 or 5173**

```
Get-NetTCPConnection -LocalPort 5000 -State Listen
Get-NetTCPConnection -LocalPort 5173 -State Listen
Get-Process -Id (Get-NetTCPConnection -LocalPort 5000 -State Listen).OwningProcess
```

**Stop a process by id**

```
Stop-Process -Id 12345
```

Replace `12345` with the PID returned above. Do not kill unrelated processes blindly.

## 2.10 Common failure → command

| Symptom | First command or check |
|---|---|
| `npm.ps1 cannot be loaded` | Use `npm.cmd`. |
| MongoDB timeout / whitelist hint | Check `MONGO_URI` and whitelist the current IP in Atlas. Startup exits after 5 seconds. |
| Frontend loads but API calls 404 | Confirm backend is on 5000 and Vite proxy is active in `vite.config.js`. |
| `EADDRINUSE` | `Get-NetTCPConnection -LocalPort 5000 -State Listen`, then stop the old Node process. |
| Blank page after refresh in production | Rebuild frontend and confirm `frontend/dist/index.html` exists; backend serves it. |
| AI replies fail | Check `GROQ_API_KEY`; inspect the backend console for the Groq error. |
| Chatbot fails only on deployed site | It calls `http://localhost:5000` directly. Replace with a relative `/api` URL or environment variable. |
| Paid test opens without Razorpay | Expected: the code falls back to the manual UPI “simulate” flow when Razorpay keys are absent. |

## 2.11 Commands used to produce this guide

The document build is separate from the application. On this machine, Chrome headless is used for PDF, and Node scripts using `html-to-docx` and an HTML-based Word document are used for DOCX/DOC. The working files live in the system temp directory, not in the project.

**Representative PDF render command**

```
& "C:\Program Files\Google\Chrome\Application\chrome.exe" --headless=new --disable-gpu --no-sandbox --no-pdf-header-footer --print-to-pdf="C:\path\output.pdf" "file:///C:/path/input.html"
```

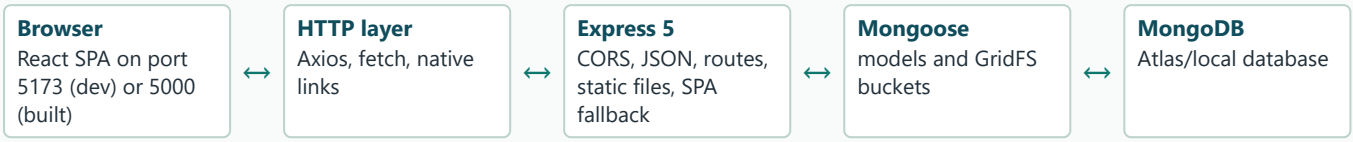
CHAPTER 03

# Architecture and Request Lifecycle

> A single request crosses several boundaries: browser component → Axios → Vite proxy or Express static server → middleware → route → controller/route handler → Mongoose → MongoDB → JSON → React state.

## 3.1 High-level architecture

**RUNTIME ARCHITECTURE**




External services sit beside this chain: Groq for language-model responses, Google Translate TTS for Hindi audio, Razorpay for card/UPI checkout, Wandbox for code compilation, and SMTP for password-reset email.

## 3.2 Backend middleware order

`backend/src/index.js` is short but its order matters:

1. `cors()` allows the dev frontend origin to call the API.
2. `express.json()` parses JSON bodies for login, tests, AI and payment routes.
3. Eight API routers are mounted under `/api/...`.
4. `/uploads` serves disk files and forces inline display for PDFs.
5. A second `/uploads` handler returns JSON 404 for missing files.
6. `express.static(frontendDist)` serves the built React assets.
7. `app.get("/{*path}")` returns `index.html` for client-side routes.
8. The database connects, then the server listens on `PORT || 5000`.

**STARTUP ORDERING DETAIL**

The server calls `connectDB()` and `app.listen()` without awaiting the database. If MongoDB is unavailable, the connection function exits the process after its 5-second timeout, but the listener has already been created. In practice the process dies, yet a cleaner design is `await connectDB(); app.listen(...)`.

## 3.3 URL resolution in development vs production

| Call | Development | Production-style |
|---|---|---|
| `axios.get("/api/notes")` | Request goes to Vite at 5173; proxy forwards to 5000. | Request goes directly to Express on 5000. |
| `axios.get("http://localhost:5000/api/ai/chat")` | Works only on the same machine. | Breaks for remote clients because localhost means the client machine. |
| `<a href="/api/pyq/:id/pdf">` | Vite proxies the PDF request. | Express streams the PDF from GridFS. |
| `<img src="/uploads/qr_...png">` | Vite proxies to Express static. | Express serves the file directly. |
| React route `/mock-test` | Vite handles SPA fallback. | Express catch-all returns `index.html`. |

## 3.4 Route registry

| Mount | Responsibility | Auth pattern |
|---|---|---|
| `/api/auth` | Signup, login, profile, activity, password reset. | Mixed; profile and activity protected. |
| `/api/admin` | Stats, students, courses, purchases, manual unlock. | Mixed; several read routes are public. |

| Mount | Responsibility | Auth pattern |
|---|---|---|
| `/api/mocktest` | Tests, PDF upload, manual add, payment, submission, results. | `protect` on most routes; admin check inside upload/add. |
| `/api/notes` | List/read/upload/delete notes and stream PDFs. | Public reads; protected writes, but no admin check. |
| `/api/pyq` | List/upload/delete PYQs and stream PDFs. | Public reads; protected writes, but no admin check. |
| `/api/payment` | Public payment settings, admin settings, QR upload. | Public GET; admin-checked settings routes. |
| `/api/certificate` | Progress, issue, my certificates, read-note tracking. | All protected. |
| `/api/ai` | Chat, note explanation, text-to-speech. | None. |

## 3.5 A protected request, step by step

**EXAMPLE: OPENING /API/MOCKTEST**

**1. Component**
`MockTest.jsx` runs `useEffect` after first render.

↓

**2. Axios**
adds `Authorization: Bearer <token>` from localStorage.

↓

**3. Proxy / Express**
Vite forwards to port 5000 in development.

↓

**4. protect middleware**
Splits the header, verifies JWT, loads the user, assigns `req.user`.

↓

**5. Route handler**
Queries `MockTest` and `Purchase`, hides `correctAnswer`, computes `purchased`.

↓

**6. Response**
JSON array → Axios → `setTests` → cards rendered.

## 3.6 Where data is stored

| Data | Storage | Notes |
|---|---|---|
| Users, profiles, activity | `users` collection | Password and reset fields are `select:false`. |
| Notes metadata | `notes` collection | Extracted text stored in `content`; file id points to GridFS. |
| Note PDFs | GridFS bucket `pdfs` | Chunks and file metadata are stored in MongoDB. |
| PYQ metadata | `pyqs` collection | University, course, subject, file id. |
| PYQ PDFs | GridFS bucket `pyqs` | Separate bucket from notes. |
| Mock tests | `mocktests` collection | Embedded question and option arrays, including correct answers. |
| Results | `results` collection | Full per-question review data and computed score. |
| Purchases | `purchases` collection | Unique index on `userId + testId`. |
| Certificates | `certificates` collection | Unique human-readable certificate id. |
| Payment settings | `settings` collection | One document keyed `payments`. |
| QR images | `backend/uploads` | Disk storage, URL exposed as `/uploads/...`. |

## 3.7 Architectural strengths and risks

**Strengths**

Clear folder split; mounted routers; shared auth middleware; embedded questions simplify scoring; GridFS avoids external object storage; SPA fallback makes deployment simple.

**Risks**

No central authorisation policy; no request validation layer; public AI endpoints; large route files; direct localhost URLs; static course lists separate from the database.

The next part examines each backend file in execution order. When you present the project, use the request lifecycle above as the “spine” and attach each feature to it.

II

# Backend Code Analysis

The Express application from boot to feature routes: database connection, authentication, user data, admin operations, file storage, mock tests, certificates, payments and AI integrations.

CHAPTER 04

# Server Bootstrap and MongoDB

> Two small files define the entire runtime foundation: `src/index.js` creates the Express application and mounts every feature, while `src/db.js` owns the MongoDB connection and failure policy.

## 4.1 The Express entry point

`backend/src/index.js` imports `express`, `dotenv`, `cors`, `path`, the database connector and all eight route modules. It computes three directory anchors:

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const rootDir = path.resolve(__dirname, "..");
```

`__dirname` is `backend/src`; `rootDir` is `backend`. Those anchors are used for `backend/uploads` and `frontend/dist`. Because the package declares `"type": "module"`, ES module syntax is used throughout and `__dirname` must be reconstructed manually.

### Middleware and routes

```
const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/auth", authroutes);
app.use("/api/admin", adminRoutes);
app.use("/api/mocktest", mockTestRoutes);
app.use("/api/notes", notesRoutes);
app.use("/api/pyq", pyqRoutes);
app.use("/api/ai", aiRoutes);
app.use("/api/payment", paymentRoutes);
app.use("/api/certificate", certificateRoutes);
```

There is no global error handler, request logger, rate limiter, helmet security header layer, or body-size limit beyond Express defaults. JSON parsing is global, so every JSON route depends on it. The CORS configuration is wide open (`cors()` with no origin allowlist), which is convenient during development but should be restricted in production.

### Static files and SPA fallback

```
const uploadsDir = path.join(rootDir, "uploads");
app.use("/uploads", express.static(uploadsDir, {
  setHeaders: (res, p) => {
    if (p.endsWith(".pdf")) res.set("Content-Disposition", "inline");
  }
}));
app.use("/uploads", (req, res) => {
  res.status(404).json({ message: "File not found" });
});

const frontendDist = path.resolve(rootDir, "..", "frontend", "dist");
app.use(express.static(frontendDist));
app.get("/{*path}", (req, res) => {
  res.sendFile(path.join(frontendDist, "index.html"));
});
```

The first `/uploads` handler serves real files. The second is a deliberate 404 fallback so a missing upload returns JSON rather than falling through to the React application. The SPA catch-all must come after API routes; otherwise API paths would return HTML.

> **EXPRESS 5 SYNTAX**
> `app.get("/{*path}", ...)` is Express 5 wildcard syntax. In Express 4 the equivalent would be `app.get("*", ...)`. This is one reason the project requires Express 5.

### Startup sequence

```
const port = process.env.PORT || 5000;
connectDB();
app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

The database connection is started but not awaited. This keeps startup code compact but has a lifecycle weakness: the HTTP listener can be created before the database is ready. Since `connectDB()` calls `process.exit(1)` on failure, the process still stops, but a safer pattern is:

```
await connectDB();
app.listen(port, () => console.log(`Server running on port ${port}`));
```

## 4.2 MongoDB connection logic

`backend/src/db.js` defines `connectDB`:

```
const uri = process.env.MONGO_URI;
if (!uri) throw new Error("MONGO_URI is not defined in environment variables");

const conn = await mongoose.connect(uri, {
  serverSelectionTimeoutMS: 5000,
});
```

| Line / value | Purpose |
|---|---|
| `dotenv.config()` | Loads `backend/.env` before reading the URI. |
| `MONGO_URI` check | Fails fast with a clear message if the environment is incomplete. |
| `serverSelectionTimeoutMS: 5000` | Limits the wait for a reachable server to five seconds instead of the default thirty. |
| `conn.connection.host` | Logs the host only, not credentials or the full URI. |
| `process.exit(1)` | Prevents the API from running without a database. |

### Atlas-specific error hint

If the error is a `MongooseServerSelectionError`, contains “timed out”, or mentions “whitelist”, the code prints an explicit hint about MongoDB Atlas IP whitelisting. This is useful because the most common student setup failure is not a wrong password but an unwhitelisted public IP.

## 4.3 Dependencies and why they exist

| Dependency | Used for |
|---|---|
| `express` | HTTP server, routing, static files and JSON middleware. |
| `mongoose` | Schemas, models, queries, aggregation and GridFS access. |
| `mongodb` | Native driver types and GridFS bucket access. |
| `jsonwebtoken` | Login tokens and short-lived password-reset tokens. |
| `bcryptjs` | Password hashing and OTP hashing. |
| `multer` | Multipart uploads for PDFs and QR images. |
| `pdf-parse` | Extracting text from note and mock-test PDFs. |
| `nodemailer` | Sending password-reset OTP email over SMTP. |
| `razorpay` | Creating real payment orders when keys are configured. |

| Dependency | Used for |
| --- | --- |
| `cors`, `dotenv` | Cross-origin development access and environment loading. |
| `@google/generative-ai` | Installed but not used by the current AI controller, which calls Groq directly. |

**UNUSED DEPENDENCY**

`@google/generative-ai` is listed but the implemented AI path is Groq via `fetch`. Mentioning this in a viva shows you understand the actual runtime, not just the package file.

CHAPTER 05

# Authentication, JWT and OTP

> The authentication module is the most security-conscious part of the backend. It combines bcrypt password hashing, JWT sessions, a six-digit email OTP, a resend cooldown, an attempt cap and a one-time reset token.

## 5.1 Files and responsibilities

| File | Responsibility |
|---|---|
| `routes/authRoutes.js` | Maps HTTP verbs and paths to controller functions; attaches `protect` where needed. |
| `controllers/authController.js` | Registration, login, profile updates, activity, OTP and reset logic. |
| `middleware/authMiddleware.js` | Verifies the Bearer token and loads the user. |
| `utils/generateToken.js` | Signs a 7-day JWT containing the user id. |
| `utils/mailer.js` | Builds the SMTP transport and sends the OTP email. |
| `model/user.js` | Defines identity, admin flag, stats, activity, read notes and certificates. |

## 5.2 Route map

| Method and path | Handler | Auth |
|---|---|---|
| `POST /api/auth/signup` | `register` | Public |
| `POST /api/auth/login` | `login` | Public |
| `POST /api/auth/forgot-password` | `forgotPassword` | Public |
| `POST /api/auth/verify-reset-otp` | `verifyResetOtp` | Public |
| `POST /api/auth/reset-password` | `resetPassword` | Public, requires reset token |
| `GET /api/auth/profile` | `getProfile` | `protect` |
| `PUT /api/auth/profile` | `updateProfile` | `protect` |
| `POST /api/auth/add-activity` | `addActivity` | `protect` |

## 5.3 Registration: validation before creation

`register` reads `firstName`, `middleName`, `lastName`, `email`, `password` and `phone`. It validates names, phone shape and email domain before touching the database.

### Name validation

```
const nameOnlyRegex = /^[a-zA-Z]+(?:\s[a-zA-Z]+)*$/;
if (firstName !== undefined && (!firstName.trim() || !nameOnlyRegex.test(firstName.trim()))) {
  return res.status(400).json({ message: "First name must contain only letters" });
}
```

Letters and internal single spaces are allowed. Numbers, punctuation and leading/trailing whitespace after trim fail.

### Phone validation

The frontend sends a combined string such as `+919876543210`. The backend splits it with `/^(\+\d+)(\d+)$/`, looks up the expected national number length in a large country-code table, and checks that the remainder is all digits and has the expected length.

```
const match = phone.match(/^(\+\d+)(\d+)$/);
const code = match ? match[1] : "";
const number = match ? match[2] : phone;
const expectedLen = countryPhoneLength(code);
if (!/^[0-9]+$/.test(number) || (expectedLen && number.length !== expectedLen)) {
  return res.status(400).json({ message: "Please enter a valid phone number..." });
}
```

For countries absent from the table, `expectedLen` is undefined, so only the digits check applies. This is intentionally permissive but not a full libphonenumber implementation.

### Email domain allowlist

The backend accepts only a list of well-known providers ( `gmail.com` , `yahoo.com` , `outlook.com` , `proton.me` , and others). A college or company domain is rejected even if the address is valid. This is a product decision, not a standard email validation rule.

```
const domain = email?.split("@")[1]?.toLowerCase();
if (!domain || !famousDomains.includes(domain)) {
  return res.status(400).json({ message: "Please use a well-known email provider..." });
}
```

> **INCONSISTENCY TO MENTION**
> Signup has no strong-password check on the backend. The frontend checks strength, but the controller only hashes whatever password it receives. The reset-password controller does enforce `isStrongPassword` . Production registration should use the same policy server-side.

### Creation and token

```
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
const fullName = [firstName, middleName, lastName].filter(Boolean).join(" ");
const newUser = await User.create({ ... });
...
token: generateToken(newUser._id)
```

`bcrypt.genSalt(10)` means ten cost rounds. The response includes a token and basic identity fields, so a successful signup logs the user in immediately. There is no email-verification step at signup; the OTP system exists only for password reset.

## 5.4 Login and password comparison

`User.password` is declared with `select: false` , so ordinary queries omit it. Login explicitly asks for it:

```
const user = await User.findOne({ email }).select("+password");
if (!user) return res.status(400).json({ message: "Invalid email or password" });

const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) return res.status(400).json({ message: "Invalid email or password" });
```

Using the same message for unknown email and wrong password avoids account enumeration. On success the response contains `_id` , `name` , `email` , `isAdmin` and a JWT.

### Token construction

`backend/utils/generateToken.js` :

```
jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: "7d" });
```

The payload is deliberately tiny. It contains no role and no email, so role changes take effect on the next request because `protect` reloads the user document. A seven-day expiry is convenient for students but there is no refresh token, logout revocation list, or token versioning. A stolen token remains usable until expiry unless the secret is rotated.

## 5.5 The protect middleware

`backend/middleware/authMiddleware.js` :

```
if (req.headers.authorization && req.headers.authorization.startsWith("Bearer")) {
  try {
    token = req.headers.authorization.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```
    req.user = await User.findById(decoded.id).select("-password");
    next();
  } catch (error) {
    res.status(401).json({ message: "Not authorized, token failed" });
  }
}
if (!token) {
  res.status(401).json({ message: "Not authorized, no token" });
}
```

Important details:

- The header must begin exactly with `Bearer`; a malformed header falls into the no-token branch.
- `jwt.verify` checks both signature and expiry.
- The user is reloaded from MongoDB on every protected request, so `isAdmin` and profile data are current.
- If the token is valid but the user was deleted, `req.user` is `null` and `next()` still runs. Downstream code that assumes `req.user._id` will throw a 500 rather than returning 401.
- There is no explicit token scheme validation beyond `split(" ")[1]`.

> **SUGGESTED HARDENING**
> After `findById`, check `if (!req.user) return res.status(401)...`. Also return immediately after the 401 in the catch block to make the control flow unambiguous.

## 5.6 Password reset: the full OTP state machine

This is the most detailed logic in the project. Constants:

| Constant | Value | Meaning |
|---|---|---|
| `OTP_TTL_MS` | 10 × 60 × 1000 | OTP valid for 10 minutes. |
| `OTP_RESEND_COOLDOWN_MS` | 60 × 1000 | 60-second wait before a new code. |
| `OTP_MAX_ATTEMPTS` | 5 | Five wrong tries, then the OTP is invalidated. |

### Step 1 — request a code

```
const email = String(req.body.email || "").trim().toLowerCase();
const genericMessage = "If an account exists for that email, a verification code has been sent.";
const user = await User.findOne({ email }).select(
  "+resetOtpHash +resetOtpExpires +resetOtpRequestedAt +resetOtpAttempts"
);
if (!user) return res.status(200).json({ message: genericMessage });
```

The generic response prevents attackers from discovering registered emails. The cooldown is enforced next; if the last request was under 60 seconds ago, the API returns 429 with a `retryAfter` value.

```
const otp = String(crypto.randomInt(100000, 1000000));
user.resetOtpHash = await bcrypt.hash(otp, 10);
user.resetOtpExpires = new Date(now + OTP_TTL_MS);
user.resetOtpRequestedAt = new Date(now);
user.resetOtpAttempts = 0;
await user.save();
```

The six-digit code is generated with a cryptographic random integer and only its bcrypt hash is stored. The plaintext code exists long enough to be emailed.

### Step 2 — verify the code

The controller validates a six-digit format, checks expiry, checks the attempt cap, and compares with `bcrypt.compare`. A wrong code increments `resetOtpAttempts`. An expired code or exhausted attempt count clears the OTP state.

```
const resetJti = crypto.randomBytes(24).toString("hex");
user.resetOtpHash = null;
user.resetOtpExpires = null;
```

```
user.resetOtpRequestedAt = null;
user.resetOtpAttempts = 0;
user.resetTokenHash = hashToken(resetJti);
await user.save();

const resetToken = jwt.sign(
  { id: user._id, purpose: "password-reset", jti: resetJti },
  process.env.JWT_SECRET,
  { expiresIn: "10m" }
);
```

This is a two-layer one-time token design:

- The JWT is signed, expires in ten minutes, and contains a random `jti`.
- Only the SHA-256 hash of that `jti` is stored in the user document.
- Resetting consumes the stored hash, so replaying the same token fails.
- `purpose: "password-reset"` prevents a normal login token from being accepted as a reset token.

### Step 3 — set a new password

```
if (!isStrongPassword(password)) return res.status(400).json({ ... });
decoded = jwt.verify(resetToken, process.env.JWT_SECRET);
if (decoded.purpose !== "password-reset" || !decoded.id || !decoded.jti) ...
if (!user.resetTokenHash || user.resetTokenHash !== hashToken(decoded.jti)) ...
user.password = await bcrypt.hash(password, salt);
user.resetTokenHash = null;
```

The strong-password rule is: at least 8 characters, at least one letter, at least one number, and at least one non-alphanumeric character.

## 5.7 Email delivery and the development OTP

`utils/mailer.js` checks whether `SMTP_HOST`, `SMTP_USER` and `SMTP_PASS` exist. If not, it returns `{ delivered: false }` without throwing. The controller then logs a warning containing the OTP and may include `devOtp` in the JSON response when `NODE_ENV !== "production"`.

> **OTP LEAKAGE RISK**
>
> When SMTP is unavailable, the server writes the plaintext OTP to the console. That is acceptable for a local demo only if logs are private. More importantly, if `NODE_ENV` is not set to `production` on a deployed server, `devOtp` is returned to the client. Always set `NODE_ENV=production` and configure SMTP before deployment.

## 5.8 Authentication: what to say in the viva

"Passwords are hashed with bcrypt and never returned. Login issues a signed JWT containing the user id, valid for seven days. Protected routes verify the signature, reload the user and attach it to the request. Password reset uses a six-digit OTP stored only as a bcrypt hash, with a ten-minute expiry, a 60-second resend cooldown, a five-attempt limit, and a one-time reset JWT whose random id is hashed in the database."

CHAPTER 06

# Users, Profiles and Activity

> The user document is the hub of the platform. It stores identity, admin status, performance placeholders, recent test activity, read-note ids and issued certificate ids.

## 6.1 The User schema

| Field | Type / default | Why it exists |
|---|---|---|
| `firstName`, `lastName` | String, required | Part-wise registration. |
| `middleName` | String, default `""` | Optional name part. |
| `name` | String, required | Display name and certificate name. |
| `email` | String, unique, required | Login identifier. |
| `phone` | String, default `""` | Profile and certificate data. |
| `password` | String, required, `select:false` | bcrypt hash; excluded from normal queries. |
| `resetOtpHash`, `resetOtpExpires`, `resetOtpRequestedAt`, `resetOtpAttempts`, `resetTokenHash` | Hidden reset state | OTP lifecycle and replay protection. |
| `isAdmin` | Boolean, default false | Admin checks in protected routes and UI. |
| `studentId` | String, default `""` | Optional institutional identifier. |
| `college` | String, default `""` | Profile and certificate field. |
| `performanceStats` | Object with defaults | Legacy summary: testsTaken, avgScore, rank, hoursLearnt. |
| `recentActivity` | Array of title, date, score, resultId | Powers profile and analytics history. |
| `readNotes` | ObjectId array → Notes | Certificate completion tracking. |
| `certificates` | ObjectId array → Certificate | Profile count and “my certificates”. |

## 6.2 getProfile

`backend/controllers/authController.js`

```
const user = await User.findById(req.user._id).select("-password");
if (!user) return res.status(404).json({ message: "User not found" });
res.status(200).json(user);
```

Because `protect` has already loaded a sanitized user, this second query is redundant but harmless. It returns reset fields as hidden because those are `select:false`. The frontend Profile and Analytics pages consume this full document.

## 6.3 updateProfile

```
user.name = req.body.name || user.name;
user.studentId = req.body.studentId || user.studentId;
user.college = req.body.college || user.college;
const updatedUser = await user.save();
```

Only three fields can be updated. Email, phone, password and `isAdmin` cannot be changed here, which is good. The `||` pattern means an empty string does not clear a field. The Profile page currently sends only `college`, although the endpoint supports name and studentId.

> **MASS-ASSIGNMENT AVOIDED**
> Using explicit field assignments instead of `Object.assign(user, req.body)` prevents a student from setting `isAdmin: true` through the profile endpoint. This is a security-positive design choice worth mentioning.

## 6.4 addActivity

After a test submission, the frontend calls this endpoint to add a history row:

```
user.recentActivity.push({
  title,
  date: new Date(),
  score,
  resultId: resultId || null,
});
user.performanceStats.testsTaken += 1;
user.markModified("performanceStats");
await user.save();
```

What it actually updates:

- **recentActivity:** appends a row used by Profile and Analytics.
- **testsTaken:** increments by one.
- **avgScore, rank, hoursLearnt:** untouched. They remain at their defaults unless some other code changes them.

## 6.5 Why Analytics still works

`PerformanceAnalytics.jsx` does not trust `avgScore`. It parses `recentActivity[].score` strings like `"28 / 40"`, sums obtained and maximum marks, and computes its own accuracy. This is why the page shows meaningful data even though the stored summary is stale.

> **BUG / DESIGN SMELL**
>
> `markModified("performanceStats")` is unnecessary because the nested field was directly mutated and Mongoose tracks it. More importantly, the endpoint does not update `avgScore`, `rank` or `hoursLearnt`. A stronger design would compute these values from the `Result` collection instead of duplicating them in the user document.

## 6.6 Profile page data flow

**Parallel load**
Profile calls `GET /api/auth/profile` and `GET /api/certificate/mine`.

↓

**Render identity**
Initials, role badge, email, student id, college.

↓

**Render summary**
Tests taken, average score, rank, hours learnt, certificate count.

↓

**Recent activity**
Each row can navigate to `/test-review/:resultId`.

The college editor sends `PUT /api/auth/profile` with `{ college }`. After success it shows a message that the certificate will update. That is only partly true: existing issued certificates store a snapshot of the college at issue time; the profile change does not rewrite already-created certificates.

## 6.7 Registration and login from the frontend

`signUp.jsx` sends JSON to `/api/auth/signup`, stores `token` and a compact `user` object in localStorage, then redirects to `/`. It validates names, email, phone, password and confirmation before the request.

`login.jsx` calls `/api/auth/login`, then `localStorage.clear()`, stores only the new token, and redirects. It does not store the user object. This is inconsistent with signup and with `AdminLogin`, but protected pages do not depend on a stored user object.

**FRONTEND PASSWORD RULE MISMATCH**

The login screen only requires six characters, the signup screen has its own strength meter, and the backend reset flow requires eight characters plus a letter, number and special character. The registration API itself does not enforce the strong rule. Standardise the policy across all three layers.

CHAPTER 07

# Admin APIs and Dashboard Data

> Admin functionality is split between `adminRoutes.js` for statistics and purchases, content routes inside notes/PYQ/mock-test modules, and a large single-page dashboard that consumes them.

## 7.1 Admin route map

| Route | Protection | Purpose |
|---|---|---|
| `GET /api/admin/purchases` | `protect` + manual `isAdmin` | List all purchases with user and test details. |
| `POST /api/admin/purchase/manual` | `protect` + manual `isAdmin` | Create or update a manual unlock. |
| `POST /api/admin/purchase/confirm` | `protect` + manual `isAdmin` | Mark a purchase `simulated`. |
| `GET /api/admin/stats` | **Public** | Dashboard counts, subject breakdown, registration trend. |
| `GET /api/admin/courses` | **Public** | Course titles. |
| `POST /api/admin/courses` | **Public** | Create a course. |
| `GET /api/admin/students` | **Public** | List non-admin students. |

**AUTHORISATION GAP**

Stats, student list, course list and course creation are completely public. Anyone who knows the URL can enumerate student names and emails or create course records. The dashboard being hidden behind `AdminRoute` does not protect the API. This is the highest-priority backend fix after payment verification.

## 7.2 Stats endpoint in detail

```
const totalStudents = await User.countDocuments();
const totalCourses = await Course.countDocuments();
const totalNotes = await Notes.countDocuments();
const totalLectures = await Lecture.countDocuments();
const totalTests = await MockTest.countDocuments();

const tests = await MockTest.find().select("title subject topic questions duration createdAt");
const subjectBreakdown = {};
tests.forEach(t => {
  const s = t.subject || "Uncategorized";
  if (!subjectBreakdown[s]) subjectBreakdown[s] = { tests: 0, questions: 0 };
  subjectBreakdown[s].tests += 1;
  subjectBreakdown[s].questions += (t.questions || []).length;
});
```

The counts are simple. The subject breakdown iterates every test and accumulates test and question counts by subject. This is fine for a small dataset; for scalability it should be an aggregation pipeline.

### Monthly registration trend

```
const monthlyRegistrations = await User.aggregate([
  { $match: { isAdmin: false } },
  { $group: { _id: { $dateToString: { format: "%Y-%m", date: "$createdAt" } }, count: { $sum: 1 } } },
  { $sort: { _id: 1 } },
]);
```

The aggregation groups non-admin users by year-month. The route then constructs the last six months explicitly so months with zero registrations still appear in the chart. That detail is easy to miss and worth explaining in a viva: the chart is gap-filled in Node, not in MongoDB.

## 7.3 Purchases listing

```
const purchases = await Purchase.find()
  .populate("userId", "name email")
```

```
  .populate("testId", "title subject price")
  .sort({ createdAt: -1 });
```

Two populate calls join the purchase to the student and the test. Only selected fields are returned, which avoids exposing the full user document. The route sort is newest first.

## 7.4 Manual unlock

```
const purchase = await Purchase.findOneAndUpdate(
  { userId, testId },
  { userId, testId, status: "manual", amount: test.price || 0, orderId: "manual", paymentId: "manual" },
  { upsert: true, new: true }
);
```

`upsert` means the admin can unlock a test whether or not a purchase row already exists. The unique index on `userId + testId` guarantees one row per student/test pair. Status `manual` is allowed by the schema enum.

## 7.5 Confirm purchase

This endpoint is narrower: it requires an existing `purchaseId` and changes status to `simulated`. It does not verify money movement; it is an admin assertion that the payment happened.

## 7.6 Admin dashboard data loading

`frontend/src/pages/AdminDashboard.jsx` loads multiple datasets when it mounts:

```
getStats();
fetchTests();
fetchStudents();
fetchNotes();
fetchPyqs();
fetchPurchases();
fetchPaymentSettings();
```

The dashboard sections are controlled by `activeSection`:

| | | |
|---|---|---|
| **Overview**<br>Metric cards, subject breakdown, six-month registration chart. | **Tests**<br>PDF upload or manual question builder; list of existing tests. | **Students**<br>Non-admin user list from the public students endpoint. |
| **Purchases**<br>Payment rows, manual unlock form, confirm action. | **Payment settings**<br>UPI id, phone, payee name and QR image. | **Notes / PYQ**<br>Upload forms and delete actions for content. |

## 7.7 Admin login flow

`AdminLogin.jsx` uses the same `/api/auth/login` endpoint as students, then checks `response.data.isAdmin`:

```
if (response.data.isAdmin) {
  localStorage.setItem("token", response.data.token);
  localStorage.setItem("isAdmin", "true");
  localStorage.setItem("user", JSON.stringify(response.data));
  navigate("/admin");
} else {
  setError("Access denied. You do not have administrative privileges.");
}
```

The server still issues a valid student token to a non-admin who logs in here; the frontend simply refuses to store it. A determined user could call the login API directly and use the token on student routes. That is expected. The important part is that admin-only backend routes check `req.user.isAdmin`.

## 7.8 AdminRoute and trust boundary

```
const token = localStorage.getItem("token");
const isAdmin = localStorage.getItem("isAdmin") === "true";
if (!token || !isAdmin) return <Navigate to="/admin-login" />;
```

This is a user-experience guard, not a security boundary. Anyone can open DevTools and set `isAdmin=true`. Real protection must be server-side. The current backend does check admin on purchases, payment settings, mock-test upload/add, but not on content upload/delete or stats/students/courses.

## 7.9 createAdmin.js

The script connects with `MONGO_URI`, hashes a password, then either promotes an existing user or creates a new user with `isAdmin: true`. It is a bootstrap utility, not part of the running server.

**HARD-CODED CREDENTIALS**

The email and password are written directly in the file. Do not reproduce them. Replace them with environment variables, run the script once, then remove the credentials and preferably the script from the deployed repository. Rotate the password if the file has ever been shared.

CHAPTER 08

# Notes, PDF Extraction and GridFS

> Notes have two lives: the original PDF is stored in MongoDB GridFS, while extracted text is stored in the note document so the frontend can render a readable web page with headings and progress tracking.

## 8.1 The Notes model

| Field | Meaning |
|---|---|
| `title` | Required display title. |
| `description` | Short summary shown in the viewer. |
| `course` | BCA, BBA or BIOTECH; used for filtering and certificates. |
| `subject` | Subject label used by the UI and certificate progress. |
| `fileId` | GridFS file id in the `pdfs` bucket. |
| `pdfUrl` | Legacy disk filename alternative. |
| `content` | Extracted text used by `NoteViewer`. |
| `hasContent` | Boolean shortcut indicating whether web content exists. |
| `type` | `"pdf"` or `"web"`. |

## 8.2 Route map

| Route | Auth | Behaviour |
|---|---|---|
| `POST /api/notes/upload` | `protect` | Create a web note or a PDF-backed note. |
| `GET /api/notes` | Public | List notes, optionally filter by course and subject; excludes `content`. |
| `GET /api/notes/:id` | Public | Return the full note; lazily extract PDF text if needed. |
| `GET /api/notes/:id/pdf` | Public | Stream the PDF inline from GridFS or disk. |
| `DELETE /api/notes/:id` | `protect` | Delete the GridFS/disk file and the note document. |

**MISSING ADMIN CHECK**

Any logged-in student can upload or delete notes. The route uses `protect`, but it never checks `req.user.isAdmin`. A student token is enough to modify or erase course content. This should be fixed with an `isAdmin` check or a dedicated admin middleware.

## 8.3 Upload paths

`multer.memoryStorage()` keeps the file in memory, which is suitable for moderate PDFs because the buffer is immediately written to GridFS. The upload route branches:

```
if (!req.file && !content) return 400;
if (!req.file) {
  const note = new Notes({ title, description, course, subject, content, hasContent: !!content, type: "web" });
  await note.save();
  return res.status(201).json(note);
}
const extractedText = await extractPdfText(req.file.buffer);
const uploadStream = bucket.openUploadStream(req.file.originalname, {
  metadata: { title, course, subject },
});
...
const note = new Notes({
  ..., content: content || extractedText || undefined,
  hasContent: !!(content || extractedText),
  type: "pdf",
  fileId: uploadStream.id,
});
```

So a note can be:

- **Pure web note:** no file, supplied `content`.
- **PDF note:** file stored in GridFS, text extracted into `content`.
- **PDF note with manual content:** supplied content wins over extraction.

## 8.4 extractPdfText

```
async function extractPdfText(buffer) {
  try {
    const parser = new PDFParse(new Uint8Array(buffer));
    await parser.load();
    const result = await parser.getText();
    const text = (result.pages || []).map((p) => p.text || "").join("\n");
    return text.trim() || "";
  } catch (error) {
    console.error("PDF text extraction failed:", error.message);
    return "";
  }
}
```

Extraction failure is non-fatal: the PDF is still stored, but `hasContent` is false and the viewer says no online content is available. This is a sensible resilience choice. The parser is `pdf-parse` v2, imported as a named export in this file.

## 8.5 GridFS buckets

```
const getBucket = () =>
  new mongoose.mongo.GridFSBucket(mongoose.connection.db, { bucketName: "pdfs" });
```

Each call creates a bucket handle against the active connection. GridFS splits files into chunks, so a PDF can exceed MongoDB's 16 MB document limit. The file metadata keeps the original name and title/course/subject.

> **TWO BUCKETS, ONE PATTERN**
> Notes use `pdfs`; PYQs use `pyqs`. Keeping them separate makes deletion and inspection simpler and avoids filename collisions between the two content types.

## 8.6 Listing and filtering

```
const filter = {};
if (req.query.course) filter.course = req.query.course;
if (req.query.subject) filter.subject = req.query.subject;
const notes = await Notes.find(filter).select("-content");
```

Excluding `content` keeps list responses small: a long PDF can produce thousands of characters. The filter is exact and case-sensitive; the frontend must send values exactly as stored. PYQ filtering, by contrast, is case-insensitive.

## 8.7 Lazy extraction on first view

Older notes may have a `fileId` but no `content`. When `GET /api/notes/:id` sees that condition, it downloads the PDF from GridFS, extracts the text, saves it, and returns the updated note:

```
if (!note.content && note.fileId) {
  const chunks = [];
  const downloadStream = getBucket().openDownloadStream(note.fileId);
  for await (const chunk of downloadStream) chunks.push(chunk);
  const text = await extractPdfText(Buffer.concat(chunks));
  if (text) {
    note.content = text;
    note.hasContent = true;
    await note.save();
  }
}
```

This is a lazy migration strategy. The first request is slower; subsequent requests read the stored text. It also means a read-only GET performs a write, which is acceptable here but should be noted in the design discussion.

## 8.8 PDF streaming and deletion

```
res.set("Content-Type", "application/pdf");
res.set("Content-Disposition", "inline");
if (note.fileId) {
  const downloadStream = getBucket().openDownloadStream(note.fileId);
  downloadStream.on("error", ...);
  downloadStream.pipe(res);
} else if (note.pdfUrl) {
  res.sendFile(path.join(uploadDir, note.pdfUrl));
} else {
  res.status(404).json({ message: "File not found" });
}
```

The PDF is served inline, so the browser opens its viewer instead of forcing a download. Deletion removes the GridFS file first, then the note document. Missing GridFS files are ignored so the metadata can still be cleaned up.

## 8.9 Input validation gaps

| Gap | Consequence |
|---|---|
| No MIME/type filter in Multer | A non-PDF file can be uploaded; extraction fails, but the file remains. |
| No file-size limit | A very large upload can consume memory and GridFS space. |
| No admin check | Any authenticated student can write or delete content. |
| No schema required fields | Only `title` is checked in the route; course/subject may be missing. |
| Error response is a raw object | `res.status(500).json(error)` may expose driver internals. |

**RECOMMENDED FIX**

Wrap upload/delete with `protect` + `isAdmin`, add `fileFilter` for `application/pdf`, set `limits: { fileSize: 25 * 1024 * 1024 }`, validate required metadata, and return a generic `{ message: "Upload failed" }` in production.

CHAPTER 09

# Previous Year Questions

> PYQs follow the same GridFS idea as notes but with university metadata. The frontend maintains a static catalogue of universities and subjects, then matches API records by a normalised course/subject key.

## 9.1 The PYQ model

| Field | Purpose |
|---|---|
| `universityId` | Short id such as `ranchi`, `vbu`, `dspmu`. |
| `universityName` | Human-readable university name. |
| `course` | `bca`, `bba` or `biotech`. |
| `subject` | Subject name from the static catalogue. |
| `fileId` | GridFS id in the `pyqs` bucket. |
| `pdfUrl` | Optional legacy disk path. |

## 9.2 Route map

| Route | Auth | Behaviour |
|---|---|---|
| `POST /api/pyq/upload` | `protect` | Store PDF in GridFS and create metadata. |
| `GET /api/pyq` | Public | Filter by universityId, course, subject; case-insensitive exact match. |
| `GET /api/pyq/:id/pdf` | Public | Stream PDF inline. |
| `DELETE /api/pyq/:id` | `protect` | Delete file and metadata. |

## 9.3 Upload logic

```
const { universityId, universityName, course, subject } = req.body;
const uploadStream = getBucket().openUploadStream(req.file.originalname, {
  metadata: { universityName, course, subject },
});
...
const pyq = new PYQ({ universityId, universityName, course, subject, fileId: uploadStream.id });
```

Unlike notes, PYQ upload assumes `req.file` exists. If a request is sent without a file, `req.file.originalname` throws and the catch returns 500. There is no validation for missing metadata.

**SAME AUTHORISATION GAP**

The upload and delete routes only use `protect`. Any logged-in student can upload or delete previous-year papers. The admin dashboard is the intended client, but the API does not enforce that intention.

## 9.4 Case-insensitive filtering

```
const caseInsensitive = (v) => ({
  $regex: new RegExp(`^${v.replace(/[.*+?^${}()|[\]\\]/g, "\\$&").trim()}$`, "i")
});
if (req.query.universityId) filter.universityId = caseInsensitive(req.query.universityId);
if (req.query.course) filter.course = caseInsensitive(req.query.course);
if (req.query.subject) filter.subject = caseInsensitive(req.query.subject);
```

Three details matter:

- The input is escaped with `replace(/[.*+?^${}()|[\]\\]/g, "\\$&")`, preventing regex injection.
- The regex is anchored with `^...$`, so it is an exact match ignoring case, not a partial search.
- Whitespace is trimmed inside the pattern but not before escaping; the generated pattern still works for normal input.

## 9.5 Frontend catalogue and matching

`frontend/src/pages/PYQ.jsx` contains six universities and large static arrays of BCA, BBA and Biotech subjects. It fetches records for a selected university:

```
const res = await axios.get(`${API_BASE}/pyq?universityId=${universityId}`);
const map = {};
res.data.forEach((pyq) => {
  const key = `${normalizeKey(pyq.course)}:${normalizeKey(pyq.subject)}`;
  map[key] = pyq;
});
setPyqData(map);
```

`normalizeKey` lowercases, collapses whitespace and trims. The subject list is then rendered; each subject checks `getPdfLink(course, subject)`. If a matching record exists, the button links to `/api/pyq/:id/pdf`; otherwise it shows "Not Available".

## 9.6 The university carousel

The page implements a responsive card slider without an external library:

- `visible` is 1 below 768 px and 2 otherwise.
- `index` selects the active card; the track translates with CSS `calc()`.
- Auto-advance uses a 3.5-second timeout and pauses on hover or during animation.
- Touch handlers track horizontal swipe distance; over 60 px navigates next/previous.
- Dots and arrow buttons call `goTo`, with a 450 ms animation lock.

This is a good frontend detail to demonstrate: no carousel package, just state, refs, pointer/touch events and CSS transforms.

## 9.7 Data consistency risk

The subject list in the frontend is static. If an admin uploads a PYQ with a subject that is not in `subjectsData`, it will never appear in the UI. Conversely, adding a subject to the frontend does not create a database record. The matcher is a normalised exact key, so "C Programming" and "C programming" match, but "C++" and "C Plus Plus" do not.

> **VIVA POINT**
> Explain this as a catalogue-driven UI: the frontend owns the display taxonomy, while MongoDB owns the files. The API is a lookup layer keyed by university + course + subject.

CHAPTER 10

# Mock Tests: Upload and PDF Parsing

> Mock tests are the most complex backend feature. They combine file upload, heuristic text parsing, question storage, paid access, scoring and result review.

## 10.1 MockTest schema

```
const mockTestSchema = new mongoose.Schema({
  title: { type: String, required: true },
  subject: String,
  topic: String,
  duration: Number,
  price: { type: Number, default: 0, min: 0 },
  questions: [{
    question: String,
    options: [String],
    correctAnswer: String,
  }],
  createdAt: { type: Date, default: Date.now },
});
```

Questions are embedded, not a separate collection. That makes test fetch and scoring simple, but it means the correct answer lives in the same document that students fetch. The list/detail routes explicitly project it out with `.select("-questions.correctAnswer")`.

## 10.2 Route map

| Route | Auth | Purpose |
|---|---|---|
| `POST /api/mocktest/upload-pdf` | `protect` + admin | Parse questions from an uploaded PDF and save a test. |
| `POST /api/mocktest/add` | `protect` + admin | Create a test from a JSON body. |
| `GET /api/mocktest` | `protect` | List tests without correct answers and annotate purchase state. |
| `GET /api/mocktest/results/mine` | `protect` | All results for the current user. |
| `GET /api/mocktest/result/:resultId` | `protect` | Result plus full question review data. |
| `GET /api/mocktest/:id` | `protect` | One test if the user has access; correct answers hidden. |
| `POST /api/mocktest/:id/order` | `protect` | Free/paid access decision; Razorpay order or simulation. |
| `POST /api/mocktest/:id/simulate-pay` | `protect` | Manual UPI transaction id unlock. |
| `POST /api/mocktest/:id/verify` | `protect` | Razorpay signature verification and purchase record. |
| `POST /api/mocktest/:id/submit` | `protect` | Score answers and save a Result. |

## 10.3 PDF upload route

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads/"),
  filename: (req, file, cb) => {
    const safeName = file.originalname.replace(/[^a-zA-Z0-9.-]/g, "_");
    cb(null, Date.now() + "-" + safeName);
  },
});
const upload = multer({ storage });
```

Unlike notes, mock-test PDFs go to disk. The filename is prefixed with a timestamp and sanitised to a safe character set. The route then reads the file, parses it, creates a test, and returns it.

```
const dataBuffer = fs.readFileSync(req.file.path);
const parser = new PDFParse(new Uint8Array(dataBuffer));
await parser.load();
```

```
const result = await parser.getText();
fullText = result.pages.map(p => p.text).join("\n");
const questions = parseQuestions(fullText);
```

> **UPLOADED PDF IS NOT DELETED**
> After parsing, the disk file remains in `backend/uploads`. Over time this accumulates orphaned files. A cleanup step (`fs.unlink` after successful save or on error) would be appropriate.

## 10.4 parseQuestions: the core algorithm

The parser expects plain text in a predictable exam format:

```
1. What is the capital of India?
(A) Mumbai
(B) New Delhi
(C) Kolkata
(D) Chennai
Answer: B
```

It first normalises lines:

```
const lines = text.split('\n').map(l => l.trim()).filter(l => l.length > 0);
let currentQuestion = null;
```

### Question detection

```
const qMatch = line.match(/^(?:Q)?(\d+)[\.\)]\s+(.*)/i);
if (qMatch) {
  if (currentQuestion) questions.push(currentQuestion);
  currentQuestion = { question: qMatch[2], options: [], correctAnswer: "" };
  return;
}
```

Accepted starts include `1.`, `1)`, `Q1.` and `q1)`. When a new question begins, the previous one is pushed.

### Multiple options on one line

```
const regex = /\(?([A-Da-d])\)?[\.\)]\s+/g;
```

The parser finds all option markers in a line. If there is more than one, it slices the text between markers. This supports both:

```
(A) Mumbai  (B) New Delhi  (C) Kolkata  (D) Chennai
```

and each option on its own line. It caps options at four.

### Answer detection

```
const aMatch = line.match(/^(?:Ans|Answer|Correct)[:\s]+\(?([A-Da-d])\)?/i);
const idx = "ABCD".indexOf(ansVal);
if (idx >= 0 && currentQuestion.options[idx]) {
  currentQuestion.correctAnswer = currentQuestion.options[idx];
} else {
  currentQuestion.correctAnswer = ansVal;
}
```

The stored correct answer is normally the option *text*, not the letter. That matters for scoring, which compares selected option text with `q.correctAnswer`. If the answer appears before options are parsed, the letter is stored instead and scoring will not match.

### Continuation lines

```
if (currentQuestion) {
  if (currentQuestion.options.length === 0) {
    currentQuestion.question += " " + line;
  } else {
    const lastIdx = currentQuestion.options.length - 1;
    currentQuestion.options[lastIdx] += " " + line;
  }
}
```

Wrapped question or option text is appended to the current field. This is a heuristic: a stray heading after the options can be appended to the last option.

## 10.5 Parser limitations

| Input pattern | Result |
|---|---|
| Options labelled `1)` to `4)` | Not recognised; only A–D are supported. |
| Answer written as full text without a letter | Not recognised by the answer regex. |
| Two-column PDF text | Lines can interleave, producing malformed questions. |
| Scanned/image PDF | No text layer, so zero questions are parsed. |
| Answer key at the end of the document | The parser expects answers adjacent to each question. |
| More than four options | Extra options are ignored once four are collected. |

**WHY THIS IS ACCEPTABLE**

A deterministic parser keeps the project dependency-light and works for a controlled PDF template. A production system would use a structured import format (CSV/JSON) or an admin review step before publishing parsed questions.

## 10.6 Manual test creation

```
const test = await MockTest.create(req.body);
```

The admin dashboard builds a JSON object with title, subject, topic, duration, price and an array of questions. Because `MockTest.create` receives the body directly, any schema field can be supplied; fields outside the schema are ignored by Mongoose. There is no validation that each question has four options or a correct answer.

## 10.7 Security checks in this chapter

- Upload and manual add correctly check `req.user.isAdmin`.
- PDF upload does not validate MIME type or size.
- The disk filename is sanitised, reducing path traversal risk.
- The parsing route returns the full created test, including correct answers, to the admin client. That is acceptable because the caller is an admin.
- The uploaded PDF remains on disk.

CHAPTER 11

# Mock Tests: Access, Payment and Scoring

> This chapter follows a paid test from the list screen to the final result. The important ideas are purchase-based access, Razorpay signature verification, a manual UPI fallback, and server-side scoring with +4 for correct and −1 for wrong.

## 11.1 The Purchase model

| Field | Purpose |
|---|---|
| `userId` | Student who bought access. |
| `testId` | Mock test being unlocked. |
| `orderId` | Razorpay order id, `sim_...`, or `manual`. |
| `paymentId` | Razorpay payment id, UPI transaction id, or `manual`. |
| `amount` | Test price at purchase time. |
| `status` | `pending`, `paid`, `manual` or `simulated`. |

The compound unique index prevents duplicate rows:

```
purchaseSchema.index({ userId: 1, testId: 1 }, { unique: true });
```

## 11.2 Access predicate

```
const hasAccess = async (userId, test) => {
  if (!test) return false;
  const price = test.price || 0;
  if (price <= 0) return true;
  const purchase = await Purchase.findOne({ userId, testId: test._id });
  return Boolean(purchase);
};
```

This is deliberately simple: free tests are always accessible; paid tests require any purchase row. It does **not** inspect `status`. A row with `pending` would still unlock the test. In the current flow, pending rows are rarely created, but the predicate should ideally require `status: { $in: ["paid", "manual", "simulated"] }`.

## 11.3 Listing tests

```
const tests = await MockTest.find().select("-questions.correctAnswer");
const purchases = await Purchase.find({ userId: req.user._id }).select("testId status");
const purchasedIds = new Set(purchases.map(p => p.testId.toString()));
const result = tests.map(t => ({
  ...t.toObject(),
  price: t.price || 0,
  purchased: (t.price || 0) <= 0 || purchasedIds.has(t._id.toString()),
  paymentPending: false,
}));
```

The projection hides every correct answer but keeps questions and options so the card can show the question count. The frontend receives `questions.length` and renders price/status. The `paymentPending` field is always false, a placeholder rather than a real state.

## 11.4 Free purchase path

`POST /:id/order` checks free and already-owned tests before contacting Razorpay:

```
const price = test.price || 0;
if (price <= 0) return res.json({ free: true, purchased: true, testId: test._id });
if (await hasAccess(req.user._id, test)) return res.json({ purchased: true, testId: test._id });
```

For a free test, no purchase document is created. Access is granted by price alone.

## 11.5 Razorpay order creation

```
const order = await razorpay.orders.create({
  amount: price * 100,
  currency: "INR",
  receipt: `test_${test._id}_${req.user._id}`,
  notes: { testId: test._id.toString(), userId: req.user._id.toString() },
});
```

Razorpay expects the amount in paise, so rupees are multiplied by 100. The response includes the public key id, amount in rupees, currency and test id. The frontend loads the Razorpay checkout script dynamically and opens it with the order.

## 11.6 Razorpay verification

```
const expected = crypto
  .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
  .update(`${orderId}|${paymentId}`)
  .digest("hex");
if (expected !== signature) {
  return res.status(400).json({ message: "Payment verification failed" });
}
```

This is the standard Razorpay signature check. It proves the order/payment pair was signed by the server's secret. On success, a purchase row is upserted with `status: "paid"`.

> **AMOUNT IS NOT RE-VERIFIED**
> The route trusts the test price in the database and does not fetch the Razorpay payment to compare its amount or status. Signature verification is strong, but a production implementation should also query Razorpay for the payment and confirm the captured amount, currency and order id.

## 11.7 Manual UPI fallback

When Razorpay keys are absent, `/order` returns `{ simulate: true, amount, currency, testId }`. The frontend opens a UPI modal with the admin's QR and UPI id, asks the student for a transaction id (UTR), and calls `/simulate-pay`.

```
if (hasRazorpayKeys) return res.status(403)...;
const transactionId = (req.body?.transactionId || "").trim();
if (price > 0 && !transactionId) return res.status(400)...;
const purchase = await Purchase.findOneAndUpdate(
  { userId: req.user._id, testId: test._id },
  { ..., status: "simulated", orderId: "sim_" + crypto.randomUUID(), paymentId: transactionId || "sim_..." },
  { upsert: true, new: true }
);
```

> **THE MANUAL FLOW IS NOT PAYMENT VERIFICATION**
> Any non-empty string is accepted as a UTR and immediately unlocks the test. There is no check against a bank statement, UPI provider, or admin approval. The code calls the status `simulated` honestly, but the UI presents it as a real payment. This is acceptable only for a demonstration. A real implementation needs server-side payment verification or an admin approval queue.

## 11.8 Submission and scoring

The client sends an object keyed by question index:

```
{ "0": 2, "1": 0, "3": 1 }
```

The server loops over every test question:

```
test.questions.forEach((q, idx) => {
  if (answers[idx] !== undefined) {
    computedAttempted++;
```

```
    const selectedAnswer = q.options[answers[idx]];
    const isCorrect = selectedAnswer === q.correctAnswer;
    if (isCorrect) computedCorrect++;
    else computedWrong++;
    detailedAnswers.push({ ... });
  } else {
    detailedAnswers.push({
      questionIndex: idx, selectedOption: -1, isCorrect: false,
      correctAnswer: q.correctAnswer, selectedAnswer: null,
    });
  }
});
const computedScore = (computedCorrect * 4) - (computedWrong * 1);
```

```
score = (correct × 4) − (wrong × 1)   |   maxScore = totalQuestions × 4
```

Unattempted questions do not reduce the score. A test with 10 questions has a maximum of 40. If a student answers 5 correctly and 3 incorrectly, the score is `20 − 3 = 17`; two unattempted questions contribute nothing.

The Result document stores:

- full `answers` array with question index, selected option, correctness, correct answer and selected answer;
- `score` and `maxScore`;
- `attempted`, `correct` and `wrong`;
- `userId`, `testId`, title and timestamps.

## 11.9 Server-side scoring is a strength

The browser never receives the correct answers while taking the test, and the client cannot submit a score. The server calculates everything from the stored question set. This is the right trust boundary and should be highlighted in the viva.

## 11.10 Result listing and review

```
GET /results/mine → Result.find({ userId }).select("-answers")
GET /result/:resultId → Result.findOne({ _id, userId })
GET /:id → MockTest.findById(...).select("-questions.correctAnswer")
```

The “mine” list excludes per-question answers to keep the payload small. The review endpoint requires both the result id and the logged-in user id, so one student cannot open another student’s review. It fetches the original test and returns questions, options and correct answers for review.

> **ORDERING BUG RISK**
> The comments in `mockTestRoutes.js` explicitly warn that `/results/mine` and `/result/:resultId` must be declared before `/:id`. Express matches in order; moving them below `/:id` would treat “results” or “result” as an id and break the routes. This is a useful Express routing detail for a viva.

## 11.11 Test submission side effects

After `/submit`, the frontend calls `/api/auth/add-activity` with the title, formatted score and result id. If that second call fails, the result still exists but the profile history lacks the row. A stronger backend would update activity inside the submit transaction or let analytics read directly from `Result`.

## 11.12 Access control summary

| Action | Backend protection |
|---|---|
| List tests | Any authenticated user; correct answers hidden. |
| Fetch one test | Authenticated + `hasAccess`; correct answers hidden. |
| Submit answers | Authenticated + `hasAccess`; scored server-side. |
| View own result | Result owner only. |
| Upload/add tests | Authenticated + `isAdmin`. |

| Action | Backend protection |
| --- | --- |
| Order/verify payment | Authenticated; purchase tied to current user. |

CHAPTER 12

# Certificates and Completion

> Certificates are generated automatically when a student has read every note and attempted every mock test for a course. The design is completion-based, not score-based.

## 12.1 Course normalisation

```
const COURSE_ALIASES = {
  BCA: "BCA", BBA: "BBA",
  BIOTECH: "BIOTECH",
  BIOTECHNOLOGY: "BIOTECH",
  BIO: "BIOTECH",
};
const COURSE_PATTERNS = {
  BCA: /BCA/i,
  BBA: /BBA/i,
  BIOTECH: /BIOTECH|BIOTECHNOLOGY|\bBIO\b/i,
};
```

The URL may say `biotechnology`, `bio` or `biotech`; all normalise to `BIOTECH`. The regex patterns then match note courses and test subjects case-insensitively.

## 12.2 Completion calculation

```
const [allNotes, allTests, results] = await Promise.all([
  Notes.find({ course: pattern }),
  MockTest.find({ subject: pattern }),
  Result.find({ userId }),
]);
const doneTestIds = new Set(results.map(r => r.testId.toString()));
const user = await User.findById(userId);
const readSet = new Set((user.readNotes || []).map(n => n.toString()));
const readNotes = allNotes.filter(n => readSet.has(n._id.toString())).length;
const doneTests = allTests.filter(t => doneTestIds.has(t._id.toString())).length;
const complete = hasContent && readNotes === totalNotes && doneTests === totalTests;
```

Key consequences:

- Only the existence of a result matters, not the score. A student can fail a test and still count it as attempted.
- `readNotes` is updated when `NoteViewer` loads a note while logged in, not when the PDF is downloaded.
- If there are no notes and no tests, `hasContent` is false, so the course cannot be completed.
- Adding new content to a course after a certificate is issued does not revoke the certificate.

## 12.3 The read-note endpoint

```
await User.findByIdAndUpdate(req.user._id, {
  $addToSet: { readNotes: note._id },
});
```

`$addToSet` prevents duplicates. The endpoint first checks that the note exists, then adds its id. It does not verify that the note's course matches anything; any note can be marked read.

## 12.4 Certificate creation

```
const cert = await Certificate.create({
  userId: req.user._id,
  certificateId: generateCertificateId(),
  course,
  studentName: req.user.name,
  college: req.user.college || "",
  phone: req.user.phone || "",
  email: req.user.email || "",
```

```
  studentId: req.user.studentId || "",
});
await User.findByIdAndUpdate(req.user._id, {
  $addToSet: { certificates: cert._id },
});
```

The certificate stores a snapshot of the student's details. Later profile edits do not change an already-issued certificate. The progress route only creates a certificate if one does not already exist for that user and course.

## 12.5 Certificate id format

```
const ymd = `${year}${MM}${DD}`;
const rand = crypto.randomBytes(4).toString("hex").toUpperCase();
return `ML-${ymd}-${rand}`;
```

```
ML-YYYYMMDD-XXXXXXXX   |   8 uppercase hexadecimal characters
```

Example shape: `ML-20261025-A1B2C3D4`. The date is the server's local date at issue time. The random suffix provides uniqueness; the schema also enforces `unique: true`.

## 12.6 Certificate routes

| Route | Auth | Behaviour |
|---|---|---|
| `POST /api/certificate/notes/:id/read` | `protect` | Mark a note as read. |
| `GET /api/certificate/progress/:course` | `protect` | Progress; issue certificate if complete. |
| `GET /api/certificate/mine` | `protect` | Populate and return the user's certificates. |
| `GET /api/certificate/:id` | `protect` | Single certificate, owner only. |

## 12.7 Certificate UI

`CertificatePage.jsx` fetches the certificate by id and the profile in parallel. It renders a printable certificate with the student's name, course, college, certificate id and issue date. `MyCertificates.jsx` calls `/mine` and lists issued certificates.

The mock-test result screen also checks certificate eligibility after submission:

```
const certRes = await axios.get(`/api/certificate/progress/${encodeURIComponent(selectedTest.subject)}`, ...);
if (certRes.data.complete && certRes.data.certificate) setCertificate(certRes.data.certificate);
```

If the final required test completes the course, the result screen shows a "Course Completed" panel and a link to the certificate.

## 12.8 Design review

**Good**

Completion is derived from content and results, not a manually set flag. Certificate ids are human-readable and unique. Ownership is checked on read.

**Needs work**

No score threshold, no transaction around certificate creation, no revalidation after new content, and no admin-visible certificate registry.

CHAPTER 13

# Payments, AI and TTS

> Three supporting integrations: admin-managed UPI settings, Groq-powered study assistance, and Google Translate text-to-speech. Each is small but has important operational and security implications.

## 13.1 Payment settings

`backend/routes/paymentRoutes.js` stores one settings document:

```
const settingSchema = new mongoose.Schema({
  key: { type: String, unique: true, default: "payments" },
  upiId: String,
  phone: String,
  payeeName: String,
  qrImage: String,
}, { timestamps: true });
```

| Route | Auth | Purpose |
|---|---|---|
| `GET /api/payment/settings` | Public | Checkout UI needs UPI id, phone, payee and QR URL. |
| `GET /api/payment/settings/admin` | `protect` + admin | Same data for the admin form. |
| `POST /api/payment/settings/admin` | `protect` + admin | Update fields and optionally replace the QR image. |

### QR replacement

Multer disk storage writes files as `qr_<timestamp>.<ext>` in `backend/uploads`. When a new QR is uploaded, the old file is deleted if it exists. The response rewrites the stored filename to a public URL:

```
qrUrl: saved.qrImage ? `/uploads/${saved.qrImage}` : ""
```

**UPLOAD VALIDATION**

The QR upload accepts any file extension and has no size limit. Restrict it to image MIME types and a small maximum size. Also validate that the extension is one of `.png`, `.jpg`, `.jpeg` or `.webp`.

## 13.2 AI chat controller

`callGroq` sends a POST to the Groq OpenAI-compatible endpoint with the model `openai/gpt-oss-20b`, a system prompt and the user message. The API key comes from `GROQ_API_KEY`.

```
body: JSON.stringify({
  model: "openai/gpt-oss-20b",
  messages: [
    { role: "system", content: systemPrompt },
    { role: "user", content: userMessage },
  ],
})
```

### Language instruction

The `language` body value selects English or Hindi. The system prompt says Hindi responses must be entirely in Devanagari and English responses in English. The user message repeats the instruction, so the model sees it twice.

### Response formatting

```
reply.replace(/\r\n/g, "\n")
  .split(/\n\s*\n/)
  .map(para => para.replace(/\n+/g, " ").replace(/\s+/g, " ").trim())
  .filter(Boolean)
  .join("\n\n")
  .trim();
```

This collapses soft line breaks inside paragraphs but preserves blank-line paragraph separation. It prevents the model from returning one sentence per line, which looks broken in the chat bubble.

### AI security and reliability

- No authentication: anyone can call `/api/ai/chat` and consume the Groq quota.
- No rate limiting or message length cap.
- Error responses include the upstream `details` string, which may expose provider errors.
- The system prompt is guidance, not a hard safety boundary. User input can attempt prompt injection.
- No conversation history is sent; each request is stateless.

## 13.3 Note explanation

`explainNote` looks for a PDF in `uploads` whose filename is the subject with spaces replaced by underscores:

```
const filename = subject.replace(/\s+/g, "_") + ".pdf";
const filepath = path.join(process.cwd(), "uploads", filename);
```

If found, it extracts up to 8,000 characters and asks the model to explain the topic. If not found, it asks the model to explain from general knowledge. The response includes `hasPdf` so the UI can indicate whether source material was used.

> **PATH COUPLING**
> This relies on `process.cwd()` being `backend`. Starting the server from the repository root would look in the wrong directory. Use an absolute path derived from `import.meta.url` instead.

## 13.4 TTS proxy

```
const langCode = lang === "hindi" ? "hi" : "en";
const url = `https://translate.google.com/translate_tts?ie=UTF-
8&q=${encodeURIComponent(text)}&tl=${langCode}&client=tw-ob`;
const response = await fetch(url, { headers: { "User-Agent": "...", "Referer": "https://translate.google.com/" }
});
res.set("Content-Type", "audio/mpeg");
res.send(Buffer.from(await response.arrayBuffer()));
```

The browser cannot call Google TTS directly because of CORS and client restrictions, so the backend proxies it. The frontend splits long text into chunks of about 180 characters at word boundaries and plays each MP3 sequentially.

> **TTS CONSTRAINTS**
> This is an unofficial endpoint. It can change, rate-limit, or block requests at any time. The proxy is unauthenticated and has no text limit, so it can be abused. For production, use a licensed TTS provider and enforce length/rate limits.

## 13.5 AI/TTS frontend URLs

Both `Chatbot.jsx` and `ChatbotPage.jsx` hardcode:

```
http://localhost:5000/api/ai/chat
http://localhost:5000/api/ai/tts
```

This works only when the browser runs on the same machine as the backend. It also bypasses the Vite proxy. Replace with `/api/ai/chat` and `/api/ai/tts`, or use an environment variable such as `import.meta.env.VITE_API_BASE`.

## 13.6 Integration summary

| Integration | Failure mode | User experience |
|---|---|---|
| Razorpay | Missing keys → simulation mode | UPI QR modal instead of checkout. |
| SMTP | Missing config → no email | OTP may be logged or returned in development. |
| Groq | Bad key/quota → 500 | Chat says “trouble connecting”. |
| Google TTS | Endpoint blocked → 500 | Hindi audio may fail; browser speech is fallback. |

| Integration | Failure mode | User experience |
| --- | --- | --- |
| Wandbox | API down → error | Compiler shows a failure message. |

III

# Frontend Code Analysis

The React application from entry point to page behaviour: routing and guards, authentication screens, the mock-test engine, the note reader, student analytics, the admin dashboard, and the secondary tools.

CHAPTER 14

# React App, Routes and Guards

> The frontend is a Vite + React single-page application. Routing is declarative, theming is context-based, and access control is split between client-side guards for navigation and server-side middleware for actual data protection.

## 14.1 Bootstrap sequence

`frontend/src/main.jsx` mounts the application:

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>
);
```

`StrictMode` runs development effects twice to expose side-effect bugs. It does not double-run production effects. `ThemeProvider` wraps the whole tree so every page and the navbar can read the same dark-mode state.

## 14.2 Theme context

```
const [darkMode, setDarkMode] = useState(true);
<ThemeContext.Provider value={{ darkMode, setDarkMode }}>
```

`App.jsx` applies `dark-theme` or `light-theme` on the outer div. CSS variables then cascade to children. The navbar toggle calls `setDarkMode(!darkMode)`. The preference is not persisted, so a refresh returns to dark mode.

## 14.3 Route table

| Path | Component | Guard |
|---|---|---|
| `/` | Home | Public |
| `/bca` , `/bca/:categorySlug` | BCA, BCACategoryPage | Public |
| `/bba` , `/bba/:categorySlug` | BBA, BbaCategoryPage | Public |
| `/biotechsylla` , `/biotechsylla/:categorySlug` | BIO, BiotechCategoryPage | Public |
| `/study-notes` | StudyNotes | Public |
| `/notes/:id` | NoteViewer | Public |
| `/recorded-lectures` | RecordedLectures | Public |
| `/about` , `/contact-us` | AboutUs, ContactUs | Public |
| `/roadmapbca` , `/bbaroadmap` | Roadmaps | Public |
| `/login` , `/signup` , `/forgot-password` | Auth pages | Public |
| `/admin-login` | AdminLogin | Public |
| `/admin` | AdminDashboard | AdminRoute |
| `/mock-test` | MockTest | ProtectedRoute |
| `/test-review/:resultId` | TestReview | ProtectedRoute |
| `/analytics` | PerformanceAnalytics | ProtectedRoute |
| `/certificates` | MyCertificates | ProtectedRoute |
| `/certificate/:id` | CertificatePage | ProtectedRoute |
| `/pyq` | PYQ | Public |
| `/chatbot` | ChatbotPage | Public |
| `/profile` | Profile | Unguarded |

**PROFILE IS NOT GUARDED**

`/profile` is public at the router level. The page itself calls `/api/auth/profile`, which returns 401 without a token, but the route should still use `ProtectedRoute` for consistency and to avoid a flash of the page shell.

## 14.4 Layout and global widget

```
<Router>
  <Navbar />
  <Routes>...</Routes>
  <Chatbot />
</Router>
```

The navbar and floating chatbot appear on every route. `Navbar` uses `NavLink` for active styling and `Link` for navigation. It contains a courses dropdown for BCA, BBA and Biotech, plus PYQ, About and Contact links.

## 14.5 Login state in the navbar

```
const isLoggedIn = !!localStorage.getItem("token");
const isAdmin = localStorage.getItem("isAdmin") === "true";
```

The navbar reads storage on every render. It does not subscribe to a storage event or React auth context, so logging in or out in another tab does not update it automatically. Logout manually clears `token`, `user` and `isAdmin`, then uses `window.location.href = "/"` to force a full reload.

## 14.6 ProtectedRoute

```
const token = localStorage.getItem("token");
return token ? children : <Navigate to="/login" />;
```

This is a navigation guard, not security. Anyone can add a fake token in DevTools and render the page. The server must still validate the JWT on every API call. The guard's real job is user experience: send anonymous visitors to the login page.

## 14.7 AdminRoute

```
const token = localStorage.getItem("token");
const isAdmin = localStorage.getItem("isAdmin") === "true";
if (!token || !isAdmin) return <Navigate to="/admin-login" />;
```

The admin flag is written by the login flow and can be edited by the user. A determined visitor can set `isAdmin=true` and open the dashboard. Backend admin endpoints must enforce `isAdmin`; some currently do not, which is discussed in the security audit.

**VIVA ANSWER**

"The frontend guard improves navigation, but the backend middleware is the real security boundary. We store the JWT in localStorage and send it in an `Authorization: Bearer` header."

## 14.8 Vite configuration

```
server: {
  proxy: {
    "/api": "http://localhost:5000",
    "/uploads": "http://localhost:5000",
  },
}
```

During development, calls to `/api/...` and `/uploads/...` are forwarded to Express. This avoids CORS configuration and keeps API URLs relative. In production, Express serves `frontend/dist` and handles the same paths, so relative URLs work there too—unless a component hardcodes `localhost`.

## 14.9 Source organisation

```
src/
  App.jsx                route table
  main.jsx               React root + ThemeProvider
  components/            ProtectedRoute, AdminRoute, Chatbot
  context/               theme context
  pages/                 route-level screens
  css_files/             page and shared CSS
  utils/                 markdown parser, formatting helpers
  firebase/              Firebase initialisation placeholder
```

## 14.10 API client patterns

Most pages use Axios and repeat three lines:

```
const token = localStorage.getItem("token");
const headers = { Authorization: `Bearer ${token}` };
axios.get("/api/...", { headers });
```

There is no shared API client, no global 401 interceptor, and no automatic logout on expired tokens. A central `api.js` would remove duplication and make token handling consistent. Two chatbot files bypass even the base path by calling `http://localhost:5000` directly.

CHAPTER 15

# Authentication Screens

> Signup, login, forgot-password and admin login are separate screens. They share a JWT-based session but store different amounts of user data in localStorage.

## 15.1 Signup flow

`frontend/src/pages/signUp.jsx` collects first name, optional middle name, last name, email, country code + phone, password, confirm password and a consent checkbox. It does not collect college, student id or course at signup. The submit handler:

1. Validates that first and last name contain letters only.
2. Validates email format and a known-provider allow-list.
3. Validates phone length for the selected country code.
4. Requires 8+ characters with a letter, a number and a special character.
5. Requires password and confirm password to match.
6. Requires the consent checkbox.
7. Posts to `/api/auth/signup` using `fetch`.
8. Stores `token` and a limited `user` JSON object.
9. Redirects to `/` with a full page load.

```
await fetch("/api/auth/signup", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ firstName, middleName, lastName, email, password, phone: fullPhone }),
});

localStorage.setItem("token", data.token);
localStorage.setItem("user", JSON.stringify({
  name: data.name, firstName: data.firstName, lastName: data.lastName,
  email: data.email, _id: data._id, isAdmin: data.isAdmin || false,
}));
```

Notice that `isAdmin` is stored inside the `user` object, not as the separate `isAdmin` key used by the navbar and `AdminRoute`. A normal signup therefore does not make the admin UI appear.

## 15.2 Email-domain allow-list

Both signup and login contain a long `famousDomains` array: Gmail, Yahoo variants, Outlook, Hotmail, Live, Rediffmail, Proton, iCloud, AOL, Zoho, Yandex, Fastmail, GMX, Tutanota, Rocketmail, Inbox and Mail.ru.

```
const domain = email.split("@")[1].toLowerCase();
if (!famousDomains.includes(domain)) {
  alert("Please use a well-known email provider (Gmail, Yahoo, Outlook, etc.)");
}
```

This blocks legitimate college and company addresses. It also duplicates the backend's disposable-domain check. Keep format validation on the client and make the server the single authority for allowed domains.

## 15.3 Login flow

The login form validates email format, the same provider allow-list, and a password of at least six characters. It posts to `/api/auth/login`, then:

```
localStorage.clear();
localStorage.setItem("token", response.data.token);
window.location.href = "/";
```

Normal login stores only the token. It does not store `user` or `isAdmin`, and it clears any previous values. The navbar still treats the session as logged in because it checks only the token. Profile data is refetched from the API.

**CLIENT-SIDE PASSWORD MINIMUM**

Login accepts six characters while signup requires eight plus complexity. Existing accounts may have shorter passwords, but the mismatch is confusing. The backend remains responsible for verifying the password; client checks are only usability hints. The “Remember me” checkbox is not wired to any behaviour.

## 15.4 Forgot-password wizard

`frontend/src/pages/ForgotPassword.jsx` implements four steps:

| Step | User action | Backend call |
|---|---|---|
| 1 | Enter registered email. | `POST /api/auth/forgot-password` |
| 2 | Enter six-digit OTP. | `POST /api/auth/verify-otp` |
| 3 | Choose a new password. | Local validation only. |
| 4 | Reset and return to login. | `POST /api/auth/reset-password` |

The OTP step includes a 60-second resend countdown and supports paste/backspace keyboard handling across six separate input boxes. On successful reset, `resetToken` and `email` are cleared from component state and the user is told to sign in again.

## 15.5 OTP request behaviour

```
const res = await axios.post("/api/auth/forgot-password", { email });
setMaskedEmail(res.data.email || email);
if (res.data.devOtp) setDevOtp(res.data.devOtp);
```

In development without SMTP, the backend may return the OTP so the flow can be tested. The UI displays it in a yellow notice. This must never happen with production mail configured.

## 15.6 Admin login

`AdminLogin.jsx` uses the same `POST /api/auth/login` endpoint as the student login. After a successful response it checks `response.data.isAdmin`:

```
if (response.data.isAdmin) {
  localStorage.setItem("token", response.data.token);
  localStorage.setItem("isAdmin", "true");
  localStorage.setItem("user", JSON.stringify(response.data));
  navigate("/admin");
} else {
  setError("Access denied. You do not have administrative privileges.");
}
```

There is no separate admin route and no second admin key in this flow. The server decides whether the account is an admin; the frontend only records that decision. A non-admin receives a clear error even though the credentials were valid.

## 15.7 Firebase placeholder

`frontend/src/firebase/firebase.js` initialises Firebase Auth and a `GoogleAuthProvider`, but the config uses placeholders such as `YOUR_API_KEY` and `YOUR_PROJECT_ID`. No page imports the provider, so Google sign-in is not wired into the UI. It is safe to describe this as prepared scaffolding rather than a working feature.

## 15.8 Auth state problems

- JWT is stored in localStorage, readable by any JavaScript on the page. A successful XSS can steal it. HttpOnly cookies are safer for production.
- `isAdmin` in localStorage is user-controlled. It must never be trusted by the API.
- No token expiry check or refresh. Expired tokens produce 401s until the user manually logs out.
- The three auth paths store different keys: signup stores token + user, normal login stores only token, admin login stores all three. Pages must therefore tolerate a missing `user` object.

- The navbar state and page state are not synchronised through context.
- The “Remember me” checkbox is cosmetic; sessions persist until logout either way.
- Logout is a full page reload, which clears transient state but is heavier than client navigation.

## 15.9 What happens on refresh

The token remains in localStorage, so every route guard still treats the user as logged in. React state resets, so pages refetch their data. After a normal login there is no stored `user` JSON, which is why the navbar and profile must rely on the token and the profile endpoint rather than cached user details.

> **VIVA CONNECTION**
> “Signup and login return a JWT signed with `JWT_SECRET`. The frontend stores it and sends it as a Bearer token. The backend `protect` middleware verifies the signature and loads the current user, so role checks reflect the database rather than the localStorage flag.”

CHAPTER 16

# Mock Test Interface

> The mock-test screen is the most stateful page in the frontend. It moves through three screens—test selection, the CBT-style exam, and the result—and manages answers, a countdown, a question palette and a manual payment modal.

## 16.1 State model

| State | Meaning |
|---|---|
| `screen` | `select` , `test` , or implicitly the result screen. |
| `tests` | Tests fetched from `GET /api/mocktest` . |
| `selectedTest` | Test currently being attempted. |
| `currentIndex` | Active question index. |
| `userAnswers` | Object keyed by question index, value is the selected option index. |
| `questionStatuses` | Palette state: answered, review, not-answered. |
| `timeLeft` | Remaining seconds. |
| `resultData` / `resultId` | Summary shown after submit and the review link. |
| `searchQuery` | Filters tests by title or subject. |
| `certificate` | Set if this submission completed the course. |
| `showSimModal` , `pendingSimTest` , `simUtr` | Manual UPI state. |

## 16.2 Loading tests and profile

```
const res = await axios.get(`${API_BASE}/mocktest`, { headers: getAuthHeaders() });
setTests(res.data);
const profile = await axios.get(`${API_BASE}/auth/profile`, { headers: getAuthHeaders() });
setCandidateName(profile.data.name);
```

The profile name prefills the Razorpay checkout. If profile loading fails, the test list can still load, but the candidate name is blank.

## 16.3 Starting a test

```
const startTest = (test) => {
  setSelectedTest(test);
  setCurrentIndex(0);
  setUserAnswers({});
  setQuestionStatuses({});
  setTimeLeft((test.duration || 60) * 60);
  setCertificate(null);
  setScreen("test");
};
```

The default duration is 60 minutes when the test has no duration. Answers and palette state are reset on every start, preventing data from a previous test leaking into the next attempt.

## 16.4 Answer selection and palette

```
const handleSelectOption = (idx) => {
  setUserAnswers(prev => ({ ...prev, [currentKey]: idx }));
  setQuestionStatuses(prev => ({ ...prev, [currentKey]: "answered" }));
};
```

The palette colour is derived from this order:

1. Current question → active.
2. `answered` → green.

3. `review` → yellow.
4. `visited` or `not-answered` → neutral/red.

Clicking a palette number marks the question visited if it has no answer, then jumps to it. Marking for review stores the state and advances to the next question. “Save & Next” stores answered/not-answered and advances.

## 16.5 Countdown and auto-submit

```
const timerActive = screen === "test" && timeLeft > 0;
useEffect(() => {
  if (timerActive) {
    timerRef.current = setInterval(() => {
      setTimeLeft(prev => {
        if (prev <= 1) {
          clearInterval(timerRef.current);
          handleSubmit(true);
          return 0;
        }
        return prev - 1;
      });
    }, 1000);
  }
  return () => clearInterval(timerRef.current);
}, [timerActive]);
```

**STALE CLOSURE IN AUTO-SUBMIT**

The effect depends only on `timerActive`, not on `handleSubmit` or `userAnswers`. The interval closes over the `handleSubmit` from the render that started the timer. At timeout it may submit an empty or outdated answer set. Use a ref for the latest answers, or include a stable submit callback that reads a ref. This is a strong example of React stale-closure reasoning for the viva.

## 16.6 Manual submit

```
const answerIndices = {};
questions.forEach((q, idx) => {
  if (userAnswers[idx] !== undefined) answerIndices[idx] = userAnswers[idx];
});
await axios.post(`${API_BASE}/mocktest/${selectedTest._id}/submit`, { answers: answerIndices }, ...);
```

The frontend converts the sparse answer object into the payload the backend expects. It then adds an activity row, stores the result summary and checks certificate progress.

**FAILURE IS SHOWN AS A ZERO SCORE**

If the submit request or the follow-up activity call throws, the catch block sets `correct: 0`, `wrong: 0` and `score: 0`, then shows the result screen. The student may believe they scored zero when the request simply failed. The UI should show an error and offer retry, not fabricate a result.

## 16.7 Activity and certificate follow-ups

```
await axios.post("/api/auth/add-activity", {
  title: `Mock Test: ${selectedTest.title}`,
  score: `${score} / ${maxScore}`,
  resultId: newResultId,
}, { headers });
```

Certificate progress is checked with the test’s `subject` as the course slug. A completed course returns the certificate and the result screen renders a gold completion panel with a link to `/certificate/:id`.

## 16.8 Result screen

The result card shows total questions, attempted, correct, wrong, the score out of max, and a “Review Answers” button when a result id exists. “Restart” clears all test state and returns to selection.

```
score = correct × 4 − wrong × 1
maxScore = totalQuestions × 4
```

The result displays the server’s values, not a client-computed score, which preserves the trust boundary.

## 16.9 Test selection screen

Cards show title, subject, question count, duration and price. Free or purchased tests show “Start Test”; paid tests show “Unlock”. The search box filters by title, subject and course text. The list comes from the backend with correct answers removed.

## 16.10 Input edge cases

- No answer selected → the question is simply unattempted.
- Clearing an answer changes the palette to not-answered but leaves no key in `userAnswers`.
- Clicking “Mark for Review” without an answer still advances and marks yellow.
- The timer is client-side only; refreshing resets the clock unless the test is restarted from the beginning.
- Submitting twice is possible if the button remains enabled during the request.

> **VIVA ANSWER**
> “The browser manages the exam state for responsiveness, but it never knows the correct answers. The server recalculates the score from the submitted option indices and stores a full review record.”

CHAPTER 17

# Note Viewer and Markdown Parser

> Notes are stored as extracted text, not as rendered HTML. A heuristic parser turns that text into headings, paragraphs and lists, then the viewer adds a table of contents, scroll progress and reading time.

## 17.1 Loading a note

```
const res = await axios.get(`/api/notes/${id}`);
setNote(res.data);
```

The viewer reads the note id from the route `/notes/:id`. If the note is missing or the request fails, it shows an error panel rather than a blank page.

## 17.2 Heading extraction

```
const headings = useMemo(
  () => (note ? extractHeadings(note.content || "") : []),
  [note]
);
```

`extractHeadings` runs the same detection used by the renderer. That matters because the generated heading ids must line up with the table of contents. If the parser emitted a heading only during rendering, the TOC could point to the wrong element.

## 17.3 What counts as a heading?

| Rule | Example | Level |
|---|---|---|
| Unit/module/chapter/topic/lesson/lecture/part/section + number | `Unit 1: Introduction` | 2 |
| Numbered title with one number | `1. Data Types` | 2 if major, otherwise 3 |
| All-uppercase line with at least four letters | `OPERATORS` | 3 |
| PDF TOC unit title | `Programming Fundamentals` | 2 |

Lines longer than 90 characters, page-number lines, metadata lines and bullet lines are rejected. “Sem” lines are explicitly excluded to avoid semester headers becoming chapters.

## 17.4 PDF TOC reconstruction

`parsePdfToc` scans the first 100 lines for a bulleted unit list. It can reconstruct a hierarchy when the document has:

```
Unit 1
• Topic A
• Topic B
Unit 2
• Topic C
• Topic D
```

The parser requires at least three units and two children per unit before trusting the TOC. It then fuzzy-matches later labels to unit and section titles, using lowercase alphanumerics and a two-character prefix tolerance.

## 17.5 Rendering

```
const html = renderPlainText(note.content);
<div dangerouslySetInnerHTML={{ __html: html }} />
```

`dangerouslySetInnerHTML` is safe here **because** every piece of note text is escaped with `escapeHtml` before tags are added. The function builds only `p`, `h2`, `h3`, `ul`, `li` and label markup. It does not accept raw HTML from the note.

> **SECURITY POINT**
> If a future change interpolates note text without escaping, this becomes an XSS sink. Keep the escape-before-tag rule explicit in code review.

## 17.6 Paragraph and list handling

```
if (isBulletLine(line)) {
  flush();
  if (!list) list = [];
  list.push([stripBulletMarker(line)]);
}
if (list) {
  if (looksLikeNewBlock(line)) { flushList(); para.push(line); }
  else list[list.length - 1].push(line);
}
```

Bullet markers include common Unicode bullets, the private-use range used by PDF extractors, and the replacement character. Consecutive lines after a bullet become part of the same list item unless they look like a new block.

## 17.7 Table of contents and scrolling

```
headingEls.current = headings.map((_, i) => document.getElementById(`nv-sec-${i}`));
```

The TOC lists headings with indentation by level. Clicking an entry calls `scrollIntoView({ behavior: "smooth" })`. A scroll listener computes:

```
progress = window.scrollY / (scrollHeight - clientHeight) × 100
```

and highlights the nearest heading above the viewport. The listener is registered as passive and removed on unmount.

## 17.8 Reading time

```
const readingTime = Math.max(1, Math.round(wordCount / 200));
```

At 200 words per minute, a 600-word note shows about 3 minutes. The minimum is one minute. This is a display estimate, not tracked analytics.

## 17.9 Marking as read

```
if (token) {
  await axios.post(`/api/certificate/notes/${note._id}/read`, {}, {
    headers: { Authorization: `Bearer ${token}` },
  });
}
```

The read marker is sent when the note opens, not when the student reaches the bottom. A student can therefore mark a note complete immediately. If you wanted genuine completion, track scroll depth and send the read event only near the end.

## 17.10 Code detection

The viewer labels a note as JavaScript when the content looks like code:

```
/\bconst\s+\w+\s*=|useState|\bfunction\s+\w+\s*\(|\blet\s+\w+\s*=\s*["'{([]/
```

This controls an icon/badge only; it does not syntax-highlight. The note is still rendered as text.

## 17.11 PDF download

```
<a href={`/api/notes/${note._id}/pdf`} target="_blank" rel="noreferrer">Download PDF</a>
```

The download endpoint streams the GridFS file with a filename and `application/pdf` content type. Opening the PDF does not mark the note as read; only the viewer mount does.

## 17.12 Parser limitations

- Heuristics can misclassify short title-case lines as headings.
- Tables and images from the original PDF are flattened to text and lost.
- Two-column PDFs may interleave lines.
- The TOC parser only looks at the first 100 lines.
- No true Markdown syntax is supported; it is plain-text reconstruction.
- Very long notes render in one pass with no virtualisation.

> **VIVA ANSWER**
> "We extract text from PDFs with `pdf-parse`, store it in MongoDB, and rebuild readable structure with a heuristic parser. We escape all text before generating HTML, which keeps `dangerouslySetInnerHTML` safe. It is a presentation layer, not a full PDF renderer."

CHAPTER 18

# Analytics, Profile and Certificates

> The student area turns recent activity into charts and provides profile editing plus access to issued certificates. Most analytics are computed in the browser from a small activity array.

## 18.1 Profile page

`Profile.jsx` fetches two endpoints in parallel:

```
GET /api/auth/profile
GET /api/certificate/mine
```

The page shows initials, name, email, phone, student id, course, college and member-since date, followed by certificate count and recent activity. The review button appears only when an activity row has a `resultId`, so older rows without one cannot be reviewed.

## 18.2 College editing

```
await axios.put("/api/auth/profile", { college: college.trim() }, { headers });
```

Only the college field is editable from this page. The backend accepts a whitelist of profile fields, so sending arbitrary keys does not change roles, email or password. Saving updates the profile but does not modify already-issued certificates, which hold a snapshot.

## 18.3 Recent activity

Each mock-test submission appends:

```
{
  title: "Mock Test: <test title>",
  score: "<score> / <maxScore>",
  resultId: "<Result id>",
  date: "<timestamp>"
}
```

The profile renders the title, date, score and a review link. The backend simply pushes into the array; there is no `$slice` cap, so the activity list grows without bound. A capped array or a separate Activity collection would scale better.

## 18.4 Analytics inputs

```
const stats = user.performanceStats || {};
const activities = user.recentActivity || [];
```

The page trusts `performanceStats` for rank, hours learnt and average score, but calculates accuracy, trends, subject performance, monthly activity and streak from `recentActivity`.

## 18.5 Score parsing

```
const parts = scoreStr.split("/").map(s => parseInt(s.trim()));
return { obtained: parts[0] || 0, total: parts[1] || 100 };
```

If the score string is malformed, the total defaults to 100. This hides bad data but can distort charts. It also means a missing score is treated as zero out of 100.

## 18.6 Accuracy

$$\text{accuracy} = \text{round}\left(\frac{\Sigma \text{ obtained}}{\Sigma \text{ maximum}} \times 100\right)$$

This is a weighted average, not the mean of per-test percentages. A 40-question test therefore contributes more than a 10-question test. The result is a percentage of total marks earned, which is the more meaningful aggregate.

## 18.7 Score trend

```
activities.slice(-10).map(...percentage...)
```

The last ten activities are plotted as percentages, labelled Test 1–10. The chart also compares the last two values and prints an “Improving” or “Declining” message. Because activity is appended in submission order, the trend is chronological if the array is stored oldest-first.

## 18.8 Subject grouping

```
if (title.includes("bca")) subject = "BCA";
else if (title.includes("bba")) subject = "BBA";
else if (title.includes("biotech")) subject = "Biotech";
else subject = "General";
```

This groups by text in the activity title, not by a stored course or subject id. A title that does not contain one of those words falls into “General”. A schema-level `course` and `subject` on the activity record would be more robust.

## 18.9 Monthly activity and streak

```
monthKey = `${year}-${String(month + 1).padStart(2, "0")}`
```

The monthly chart keeps the last six month buckets. The streak counts consecutive distinct activity dates, starting from today or yesterday. Multiple tests on one day count once. If the most recent activity is older than yesterday, the streak is zero.

> **STATS THAT ARE NOT RECALCULATED**
> The dashboard displays `performanceStats.rank`, `hoursLearnt` and `avgScore`, but the codebase does not recompute them on test submission. Unless they are seeded or updated elsewhere, they remain defaults or stale. The reliable figures are the ones derived from `recentActivity`.

## 18.10 Analytics UI

The page has four tabs:

- **Overview** — summary cards and recent trend.
- **Progress** — score progression over the last ten tests.
- **Subjects** — per-subject accuracy circles.
- **History** — full activity table with score percentages.

All charts are CSS/HTML based; there is no charting library. That keeps the bundle small but limits interactivity.

## 18.11 My Certificates

```
GET /api/certificate/mine
```

The page shows a grid of certificate cards or an empty state explaining that all notes must be read and all tests attempted. Each card links to `/certificate/:id`.

## 18.12 Certificate view

`CertificatePage.jsx` requests the certificate and the profile concurrently, then uses the certificate snapshot first and the live profile as fallback. It renders a printable certificate with the course, student details, certificate id and issue date. “Print / Save as PDF” calls `window.print()`.

> **VIVA ANSWER**
> “Analytics are derived from the activity log, so the dashboard stays responsive without a separate reporting service. Rank and hours are currently placeholders; accuracy and trends are real calculations from stored scores.”

CHAPTER 19

# Admin Dashboard UI

> The admin dashboard is a single large component with a sidebar and seven sections. It combines statistics, mock-test creation, student lists, purchase management, payment settings and content uploads.

## 19.1 Sections

| Section | Purpose |
|---|---|
| Overview | Counts for students, courses, notes, lectures, tests and questions. |
| Mock Tests | Create a test manually or upload a PDF; list existing tests. |
| Students | List registered students. |
| Purchases | List purchases, manually unlock a test, confirm a pending payment. |
| Payment Settings | Set UPI id, phone, payee name and QR image. |
| Upload Notes | Upload a PDF with title, course and subject; delete notes. |
| Upload PYQ | Upload a paper by university, course and subject; delete papers. |

## 19.2 Initial load

```
useEffect(() => {
  getStats();
  fetchTests();
  fetchStudents();
  fetchNotes();
  fetchPyqs();
  fetchPurchases();
  fetchPaymentSettings();
}, []);
```

Seven requests fire on mount. Each has its own catch block, so one failure does not block the others. The trade-off is a burst of requests and no unified loading state.

## 19.3 Overview cards

Counts come from `GET /api/admin/stats`:

```
{ totalStudents, totalCourses, totalNotes, totalLectures, totalTests, totalQuestions }
```

The route is currently public, so the numbers are readable without a token. The cards are display-only; there is no drill-down.

## 19.4 Mock-test creation

The form supports two modes:

- **PDF upload** — title, subject, topic, duration, price and a PDF. The backend extracts questions with a regex heuristic.
- **Manual** — title, subject, topic, duration, price and a dynamic list of questions with four options and a correct answer.

```
const [manualQuestions, setManualQuestions] = useState([
  { question: "", options: ["", "", "", ""], correctAnswer: "" },
]);
```

The manual editor can add/remove questions and edits options immutably. On submit it posts JSON to `/api/mocktest/add` with a Bearer token.

## 19.5 PDF test upload

```
await axios.post("/api/mocktest/upload-pdf", fd, {
  headers: { Authorization: `Bearer ${token}`, "Content-Type": "multipart/form-data" },
});
```

The backend stores the PDF in `uploads`, parses text, tries to extract numbered questions and four options, and creates a test only if at least one question is found. The parsing quality depends heavily on the PDF layout.

> **NO PREVIEW STEP**
> The admin cannot review or edit parsed questions before saving. A misparsed PDF becomes a live test. A production flow should show a draft preview with editable questions and an explicit publish action.

## 19.6 Student list

```
GET /api/admin/students
```

The table shows name, email, phone and course. The route currently has no auth middleware. The frontend also calls it without a token, so adding protection requires changing both the route and this request.

## 19.7 Purchases and manual unlock

```
POST /api/admin/purchase/manual { userId, testId }
```

The admin selects a student and test, and the backend upserts a purchase with status `manual`. This is useful for cash payments or support cases. The purchase list also supports confirming a pending payment:

```
POST /api/admin/purchase/confirm { purchaseId }
```

Both routes require a valid token in the UI and should require `isAdmin` on the server. Verify that the backend route includes the admin guard; manual unlock is a high-impact action.

## 19.8 Payment settings

```
const fd = new FormData();
fd.append("upiId", payUpi);
fd.append("phone", payPhone);
fd.append("payeeName", payName);
if (payQr) fd.append("qr", payQr);
await axios.post("/api/payment/settings/admin", fd, ...);
```

The form displays the current QR, allows replacement, and saves all fields in one request. If no QR is selected, the existing one is retained. The public checkout reads the same document.

## 19.9 Notes and PYQ uploads

```
fd.append("title", notesTitle);
fd.append("description", notesDesc);
fd.append("course", notesCourse);
fd.append("subject", notesSubject);
fd.append("pdf", notesPdf);
```

Notes are parsed into text and stored with a GridFS file id. PYQs are simpler: university, course, subject and a GridFS PDF. Both lists provide “View PDF” links and delete buttons.

> **DELETE PERMISSIONS**
> The backend note/PYQ delete routes use `protect` but not an admin check. Any logged-in student could call them directly. The UI hides the buttons, but the API must enforce the role.

## 19.10 Component size

`AdminDashboard.jsx` is over 1,000 lines and mixes data fetching, form state, upload handling and presentation. It works, but it is a strong candidate for extraction into:

- `AdminLayout` and sidebar;
- one component per section;

- a shared `useAdminData` hook;
- an API module with auth headers.

## 19.11 Admin data flow

```
AdminLogin → token + isAdmin=true → /admin
         → AdminRoute checks localStorage
         → dashboard fetches stats/students/tests/notes/PYQs
         → mutations call upload/delete/add endpoints
         → local state refreshed by refetch
```

There is no cache invalidation library; each mutation calls the relevant fetch again. This is simple and predictable for a small admin panel.

> **VIVA ANSWER**
> "The admin dashboard is a single-page control panel. Mutations are followed by refetches so the counts and lists converge. The critical issue is that several admin APIs are not actually protected on the server, so the UI is not a security boundary."

CHAPTER 20

# PYQ, Lectures, Chatbot and Compiler

> Four secondary features complete the student experience: previous-year papers, recorded lectures, an AI study assistant, and a browser code runner.

## 20.1 PYQ data model

| Field | Purpose |
|---|---|
| `universityId` | Stable id such as `ranchi` or `vbu`. |
| `universityName` | Display name used in the UI. |
| `course` | BCA, BBA or Biotech. |
| `subject` | Subject name. |
| `fileId` | GridFS id for the PDF. |
| `originalName` | Original upload filename. |

## 20.2 PYQ screen

`PYQ.jsx` contains a static catalogue of six universities and three subject lists. It fetches papers for the selected university and builds a lookup:

```
const key = `${normalizeKey(pyq.course)}:${normalizeKey(pyq.subject)}`;
map[key] = pyq;
```

A subject row is available when the lookup contains a matching course/subject pair. Selecting a paper opens a modal with a download link. The carousel supports arrows, dots, hover pause, touch swipe and responsive visible counts (one card below 768px, two above).

## 20.3 Why the static catalogue exists

The list of universities and subjects is fixed in the frontend, while the PDFs are dynamic. This keeps the UI navigable even when no papers are uploaded, but it means adding a new university requires a code change. A backend catalogue would be more flexible.

## 20.4 Recorded lectures

`RecordedLectures.jsx` is a static page with:

- a local video tag using `/intro2.mp4`;
- a lecture title, instructor, duration and description;
- a playlist with four hardcoded lecture entries.

There is no backend model, no progress tracking, and no dynamic upload. It is a presentation-only feature. The playlist items are not clickable and do not change the video.

## 20.5 Chatbot widget

`components/Chatbot.jsx` renders a floating button that expands into a chat panel. It keeps:

```
messages: [{ role, content }]
input, isTyping, speakingIndex, language
```

The initial message is a greeting. Sending appends the user message, sets typing, posts to the AI endpoint, cleans the reply, and appends the bot message. Errors become a generic “trouble connecting” reply.

## 20.6 Language and speech

The language selector switches between English and Hindi. For Hindi:

```
await axios.post("http://localhost:5000/api/ai/tts",
  { text: chunk, lang: "hindi" },
  { responseType: "blob" });
```

Text is split into roughly 180-character chunks at word boundaries, each chunk is fetched as an MP3 blob, and playback is sequential. If the TTS request fails, the code falls back to the browser's `speechSynthesis` with a Hindi voice. English speech uses the browser voice directly.

> **HARDCODED HOST**
> The chatbot and TTS calls use `http://localhost:5000`, while the rest of the app uses relative `/api` URLs. This breaks when the frontend is opened from another device or deployed. The same applies to `ChatbotPage.jsx`.

## 20.7 Chat rendering

Bot text is cleaned of bold markers and simple HTML before display. Messages auto-scroll to the bottom whenever the list or typing state changes. The panel also offers a full-screen route (`/chatbot`) that reuses the same backend.

## 20.8 Code compiler

`components/CodeCompiler.jsx` supports:

| Language | Wandbox compiler |
|---|---|
| C | `gcc-12.3.0-c` |
| C++ | `gcc-12.3.0` |
| Java | `openjdk-jdk-21+35` |
| Python | `cpython-3.11.10` |
| JavaScript | `nodejs-20.17.0` |
| HTML/CSS | No compiler; rendered in an iframe |

```
const res = await fetch("https://wandbox.org/api/compile.json", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ code, compiler: lang.compiler, stdin }),
});
```

The response uses `program_output`, `compiler_error` and `program_error`. The UI shows the first available error, or "(no output)" when the program prints nothing.

## 20.9 HTML preview security

```
<iframe srcDoc={previewHtml} sandbox="allow-scripts" />
```

Omitting `allow-same-origin` gives the preview a unique opaque origin. Scripts can run inside the frame but cannot read the parent's localStorage, cookies or DOM. This is the correct sandbox choice for user-authored HTML.

> **THIRD-PARTY CODE EXECUTION**
> Compiling sends the user's code to Wandbox, an external service. There is no privacy notice or rate limit. Wandbox also may be unavailable. For a college project this is acceptable; for production, use a controlled execution service.

## 20.10 Feature maturity

| Feature | Backend | Persistence |
|---|---|---|
| PYQ | Yes, GridFS + Mongo | Dynamic papers, static catalogue. |
| Lectures | No | Static files and hardcoded playlist. |
| Chatbot | Groq proxy | No conversation history. |
| TTS | Google proxy | No caching. |

| Feature | Backend | Persistence |
| --- | --- | --- |
| Compiler | External Wandbox | No saved snippets. |

IV

# Cross-Cutting Analysis

The whole system viewed horizontally: the data model atlas, the complete API reference, end-to-end request flows, the security and quality audit, and a practical debugging playbook.

CHAPTER 21

# Data Model Atlas

> MongoDB holds ten Mongoose models plus two GridFS buckets. The core relationships are user → results, user → purchases, and result/purchase → mock test. Notes and PYQs reference GridFS files rather than storing PDF bytes in documents.

## 21.1 Model map

```
User ──< Result >── MockTest
  │                    │
  ├──< Purchase >──────┘
  ├──< Certificate
  ├── readNotes [] ──< Notes
  └── certificates [] ──< Certificate

MockTest.questions[]  embedded question + options + correctAnswer
Purchase { userId, testId }  unique pair
GridFS: pdfs (notes), pyqs (previous-year papers)
uploads/: QR image + mock-test source PDFs
```

## 21.2 User

| Field | Type / rule | Notes |
|---|---|---|
| `firstName`, `lastName` | String, required, trim | Validated as letters only. |
| `middleName` | String, default "" | Optional. |
| `name` | String, required | Concatenated full name. |
| `email` | String, unique, required | No lowercase transformer; uniqueness is case-sensitive at the DB level. |
| `phone` | String, default "" | Stored with country code. |
| `password` | String, required, `select:false` | bcrypt hash, cost 10. Never returned by profile. |
| `resetOtpHash`, `resetOtpExpires`, `resetOtpRequestedAt`, `resetOtpAttempts` | hidden fields | Password-reset state, all `select:false`. |
| `resetTokenHash` | String, hidden | SHA-256 hash of the short-lived reset JWT. |
| `isAdmin` | Boolean, default false | Server-side role flag. |
| `studentId`, `college` | String, default "" | Editable from profile. |
| `performanceStats` | Object | Defaults: testsTaken 0, avgScore "0%", rank "N/A", hoursLearnt "0h". |
| `recentActivity` | Array | title, date, score string, optional resultId. |
| `readNotes` | [ObjectId → Notes] | Completion tracking. |
| `certificates` | [ObjectId → Certificate] | Issued certificate references. |
| `createdAt`, `updatedAt` | Date | Timestamps enabled. |

**DEFAULT STATS**

`addActivity` increments `testsTaken` only. It does not recompute `avgScore`, `rank` or `hoursLearnt`, which remain at their default strings. The UI displays them anyway, so they can look real when they are placeholders.

## 21.3 MockTest

```
{
  title: String (required),
  subject: String,
  topic: String,
  duration: Number,          // minutes
```

```
  price: Number (default 0, min 0),
  questions: [{
    question: String,
    options: [String],
    correctAnswer: String
  }],
  createdAt: Date
}
```

Questions are embedded. That makes reading a test a single document fetch and keeps each test self-contained. The trade-off is a 16 MB document limit, which is far above any realistic quiz size.

## 21.4 Notes

| Field | Purpose |
|---|---|
| `title`, `description` | Display metadata. |
| `course`, `subject` | Used by filters and certificate matching. |
| `fileId` | GridFS id in the `pdfs` bucket. |
| `pdfUrl` | Legacy disk path fallback. |
| `content` | Extracted plain text for the web reader. |
| `hasContent` | True when text or web content exists. |
| `type` | `pdf` or `web`. |

Older notes may have a file but no content. The single-note route lazily extracts text on first view and saves it, so the first request can be slower.

## 21.5 PYQ

```
{
  universityId, universityName,
  course, subject,
  fileId, pdfUrl
}
```

The university and subject catalogue lives in the frontend; the database stores only uploaded papers. Queries are case-insensitive exact matches built with escaped regular expressions.

## 21.6 Result

| Field | Meaning |
|---|---|
| `userId`, `testId` | References, both required. |
| `title` | Snapshot of the test title. |
| `answers[]` | questionIndex, selectedOption, isCorrect, correctAnswer, selectedAnswer. |
| `score`, `maxScore` | Required integers. |
| `attempted`, `correct`, `wrong` | Counts, default 0. |
| `createdAt`, `updatedAt` | Attempt timestamp. |

There is no unique index on `{ userId, testId }`, so a student can create multiple results for the same test. Certificate completion only needs at least one, and analytics treat every attempt as separate activity.

## 21.7 Purchase

```
{
  userId, testId,
  orderId, paymentId,
  amount: Number,
  status: "paid" | "manual" | "simulated" | "pending"
```

```
}
index: { userId: 1, testId: 1 }, unique
```

The unique pair guarantees one access row per student/test. `findOneAndUpdate(..., { upsert: true })` makes repeat payment attempts idempotent at the row level.

## 21.8 Certificate

| Field | Meaning |
|---|---|
| `userId` | Owner reference. |
| `certificateId` | `ML-YYYYMMDD-XXXXXXXX`, unique. |
| `course`, `subject` | Normalised course; subject currently unused. |
| `studentName`, `college`, `phone`, `email`, `studentId` | Snapshot at issue time. |
| `score` | Present but not populated by the issuance route. |
| `issuedAt` | Issue date, default now. |

There is no unique `{ userId, course }` index. The route checks first, but concurrent requests could theoretically create duplicates. Add a compound unique index to make it airtight.

## 21.9 Setting

```
{ key: "payments", upiId, phone, payeeName, qrImage }
```

A single document keyed by `payments`. The admin route upserts it; the public route returns a computed `qrUrl`.

## 21.10 Course and Lecture

```
Course { title, description }
Lecture { title, description }
```

Both are intentionally minimal. Course is counted by the admin stats and exposed through `/api/admin/courses`, but signup does not assign a course. Lecture is only used for the stats count; the recorded-lectures page is static. These models are placeholders for future content management.

## 21.11 File storage

| Storage | Contents |
|---|---|
| GridFS `pdfs.files` / `pdfs.chunks` | Study-note PDFs. |
| GridFS `pyqs.files` / `pyqs.chunks` | Previous-year papers. |
| `backend/uploads` | QR image and uploaded mock-test PDFs. |

GridFS is the right choice for PDFs because it supports files larger than a Mongo document and streams them without loading everything into memory. Mock-test PDFs are written to disk because they are used as an admin reference and are not streamed back through a dedicated route.

## 21.12 Data integrity gaps

- No cascade delete: deleting a note does not remove its id from users' `readNotes`.
- No cascade delete: deleting a test leaves results and purchases pointing at a missing test.
- Activity is unbounded and duplicates summary data.
- Results are denormalised snapshots by design; later test edits do not rewrite old results.
- Certificates are snapshots by design; later profile edits do not rewrite them.

CHAPTER 22

# API Reference

> All endpoints are rooted at `/api`. "Public" means no token, "Protected" means a valid Bearer JWT, and "Admin" means a protected route that also checks `req.user.isAdmin`.

## 22.1 Authentication

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /auth/signup` | Public | Register and return a JWT. |
| `POST /auth/login` | Public | Verify password; return JWT and `isAdmin`. |
| `POST /auth/forgot-password` | Public | Generate a hashed OTP, email it, return generic message. |
| `POST /auth/verify-reset-otp` | Public | Verify OTP, return a short-lived reset JWT. |
| `POST /auth/reset-password` | Public | Consume reset JWT and set a new password. |
| `GET /auth/profile` | Protected | Current user without password. |
| `PUT /auth/profile` | Protected | Update name, studentId, college. |
| `POST /auth/add-activity` | Protected | Append an activity and increment testsTaken. |

## 22.2 Notes

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /notes/upload` | Protected | Multipart upload; PDF or JSON `content`. |
| `GET /notes?course=&subject=` | Public | List metadata without content. |
| `GET /notes/:id` | Public | Full note; lazy PDF extraction if needed. |
| `GET /notes/:id/pdf` | Public | Stream PDF inline. |
| `DELETE /notes/:id` | Protected | Delete document and GridFS file. |

## 22.3 PYQ

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /pyq/upload` | Protected | Upload a paper to GridFS. |
| `GET /pyq?universityId=&course=&subject=` | Public | List papers; filters are case-insensitive. |
| `GET /pyq/:id/pdf` | Public | Stream paper inline. |
| `DELETE /pyq/:id` | Protected | Delete paper and GridFS file. |

## 22.4 Mock tests

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /mocktest/upload-pdf` | Admin | Upload/parse a test PDF. |
| `POST /mocktest/add` | Admin | Create a manual test. |
| `GET /mocktest` | Protected | List tests with answers removed. |
| `GET /mocktest/results/mine` | Protected | Own results without answer arrays. |
| `GET /mocktest/result/:resultId` | Protected | Own result + questions + answers. |
| `GET /mocktest/:id` | Protected | Test if purchased; answers hidden. |
| `POST /mocktest/:id/order` | Protected | Free access, Razorpay order, or simulate flag. |
| `POST /mocktest/:id/simulate-pay` | Protected | Manual UTR unlock when Razorpay is absent. |
| `POST /mocktest/:id/verify` | Protected | Verify Razorpay signature and save purchase. |
| `POST /mocktest/:id/submit` | Protected | Score answers and store a result. |

> **ROUTE ORDER MATTERS**
> `/results/mine` and `/result/:resultId` are declared before `/:id`. If reordered, Express would match "results" as an id and the routes would break.

## 22.5 Certificates

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /certificate/notes/:id/read` | Protected | Add note to `readNotes`. |
| `GET /certificate/progress/:course` | Protected | Progress and possible issuance. |
| `GET /certificate/mine` | Protected | Populated certificate list. |
| `GET /certificate/:id` | Protected | Own certificate only. |

## 22.6 Admin

| Endpoint | Access | Purpose |
|---|---|---|
| `GET /admin/purchases` | Admin | All purchases with student/test summary. |
| `POST /admin/purchase/manual` | Admin | Unlock a test for a student. |
| `POST /admin/purchase/confirm` | Admin | Mark a purchase simulated/confirmed. |
| `GET /admin/stats` | **Public** | Counts, subject breakdown, registration trend. |
| `GET /admin/courses` | **Public** | Course titles. |
| `POST /admin/courses` | **Public** | Create a course. |
| `GET /admin/students` | **Public** | Student name, email, studentId, createdAt. |

The three public admin routes are a real security gap, not an intentional read-only API. They should require the same admin check as purchases.

## 22.7 Payment settings

| Endpoint | Access | Purpose |
|---|---|---|
| `GET /payment/settings` | Public | UPI id, phone, payee, QR URL. |
| `GET /payment/settings/admin` | Admin | Same data for the editor. |
| `POST /payment/settings/admin` | Admin | Update settings and optional QR. |

## 22.8 AI

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /ai/chat` | Public | Groq chat with English/Hindi prompt. |
| `POST /ai/explain` | Public | Explain a subject, optionally using a PDF. |
| `POST /ai/tts` | Public | Proxy Google Translate TTS. |

## 22.9 Common response shapes

```
// login
{ _id, name, email, isAdmin, token }

// verify Reset OTP
{ resetToken }

// mock test list item
{ _id, title, subject, topic, duration, price, questions, purchased }

// submit
{ resultId, score, maxScore, correct, wrong, totalQuestions, message }

// certificate progress
{ course, totalNotes, readNotes, totalTests, doneTests,
```

```
  hasContent, complete, certificate }

// error
{ message }
```

## 22.10 Status codes used

| Code | Typical cause |
|---|---|
| 200 | Success or idempotent no-op. |
| 201 | User, note, PYQ, test, or certificate created. |
| 400 | Validation failure or bad credentials. |
| 401 | Missing or invalid JWT. |
| 403 | Not an admin, or test not purchased. |
| 404 | Resource or file not found. |
| 500 | Unhandled server/database/provider error. |

## 22.11 Example calls

```
# login
curl -X POST http://localhost:5000/api/auth/login ^
  -H "Content-Type: application/json" ^
  -d "{\"email\":\"student@example.com\",\"password\":\"Secret123!\"}"

# authenticated test list
curl http://localhost:5000/api/mocktest ^
  -H "Authorization: Bearer <token>"

# submit answers
curl -X POST http://localhost:5000/api/mocktest/<id>/submit ^
  -H "Authorization: Bearer <token>" ^
  -H "Content-Type: application/json" ^
  -d "{\"answers\":{\"0\":2,\"1\":0}}"
```

On Windows CMD, `^` continues a command; in PowerShell use backticks or keep the request on one line.

CHAPTER 23

# End-to-End Flows

> These are the sequences to trace in a viva. Each flow lists the exact actors, requests, state changes and failure points.

## 23.1 Registration and login

```
Browser → POST /api/auth/signup
        → validate names, phone, domain
        → check duplicate email
        → bcrypt hash password (10 rounds)
        → User.create
        → sign JWT { id }
        → 201 { token, user fields }
Browser → save token (+ partial user) → redirect /
Later   → GET /api/auth/profile with Bearer token
        → protect verifies JWT → loads user → returns profile
```

**Failure points:** duplicate email, unsupported domain, MongoDB offline, weak client password, JWT secret missing. The backend does not currently call `isStrongPassword` during registration, so a direct API call can create a weak password.

## 23.2 Password reset

```
POST /auth/forgot-password { email }
→ find user (or return generic success)
→ generate 6-digit OTP
→ hash OTP with SHA-256, set expiry 10 min, attempts 0
→ send email (or log OTP when SMTP missing)
→ generic 200 response

POST /auth/verify-reset-otp { email, otp }
→ compare SHA-256 hash, check expiry and attempts
→ increment attempts on failure
→ issue short-lived reset JWT with purpose + jti
→ store hash of jti/hash token on user

POST /auth/reset-password { resetToken, password }
→ verify signature, purpose, jti hash
→ enforce strong password
→ bcrypt hash and save
→ clear OTP/reset fields
→ 200 “You can now sign in”
```

The generic forgot-password response prevents account enumeration. The OTP is never stored in plaintext. The reset token is single-use because its hash is cleared after success.

## 23.3 Reading notes and completing a course

```
Student opens /study-notes
→ GET /api/notes?course=...
→ selects note → /notes/:id
→ GET /api/notes/:id (content; lazy extraction if needed)
→ if logged in: POST /api/certificate/notes/:id/read
→ $addToSet readNotes

Student opens /mock-test
→ GET /api/mocktest
→ attempts every test → results stored

GET /api/certificate/progress/:course
→ Notes.find(course pattern)
→ MockTest.find(subject pattern)
→ Result.find(userId)
→ readNotes = notes present in user.readNotes
```

```
→ doneTests = tests with any result
→ complete = hasContent && all notes read && all tests attempted
→ if complete and no existing certificate: create + link
```

**Important:** completion does not check score. A failed attempt still counts as “attempted”. Opening a note marks it read immediately.

## 23.4 Free test attempt

```
GET /api/mocktest
→ price 0 → purchased=true
POST /api/mocktest/:id/order
→ price <= 0 → { free: true, purchased: true }
Browser startTest()
→ timer = duration × 60
→ answers stored locally
POST /api/mocktest/:id/submit
→ hasAccess true because price is 0
→ score = 4 × correct − 1 × wrong
→ Result.create
→ 200 result summary + resultId
POST /api/auth/add-activity
→ push activity, testsTaken += 1
GET /api/certificate/progress/:subject
→ maybe issue certificate
```

## 23.5 Paid test with Razorpay

```
POST /api/mocktest/:id/order
→ hasAccess false
→ razorpay.orders.create({ amount: price*100, currency: "INR" })
→ 200 { orderId, amount, currency, keyId }

Browser opens Razorpay checkout
→ user pays
→ handler receives order_id, payment_id, signature
POST /api/mocktest/:id/verify
→ HMAC-SHA256(order_id|payment_id, KEY_SECRET)
→ compare signature
→ Purchase.findOneAndUpdate(status: paid, upsert)
→ 200 purchased

POST /submit
→ hasAccess checks purchase row
→ score and store result
```

**Failure points:** missing keys, failed signature, abandoned checkout, duplicate purchase (upsert handles it), result submit failure after a successful payment.

## 23.6 Paid test with manual UPI

```
POST /order
→ no Razorpay keys → { simulate: true, amount }

Browser GET /api/payment/settings
→ show QR + UPI id
→ user pays externally and enters UTR
POST /api/mocktest/:id/simulate-pay { transactionId }
→ reject if Razorpay keys now exist
→ require non-empty UTR for paid test
→ Purchase.findOneAndUpdate(status: simulated, paymentId: UTR)
→ 200 purchased

POST /submit → score
```

This flow is a demo. The UTR is not verified against any bank or provider, so it must not be presented as real payment verification in production.

## 23.7 Answer review

```
Result screen → /test-review/:resultId
→ GET /api/mocktest/result/:resultId
→ Result.findOne({ _id, userId })
→ MockTest.findById(result.testId)
→ return { result, questions[] with correctAnswer }
→ UI compares selectedOption with correctAnswer
→ green/red/unattempted styling
```

Only the owner can load the review because the query includes `userId`. This is a good example of object-level authorization.

## 23.8 Admin publishes a note

```
Admin → /admin-login → POST /auth/login
→ response.isAdmin true → store token + isAdmin
→ /admin → AdminRoute passes

Upload Notes form → multipart POST /api/notes/upload
→ protect verifies token
→ multer keeps PDF in memory
→ extractPdfText(buffer)
→ GridFS write to pdfs bucket
→ Notes.create({ fileId, content, hasContent: true })
→ 201
→ frontend refetches notes and stats
```

There is no admin role check on note upload, only authentication. Any logged-in user can call the same endpoint.

## 23.9 Admin publishes a mock test

```
Option A: POST /mocktest/upload-pdf
→ admin check
→ save PDF to uploads
→ extract text
→ parseQuestions heuristics
→ reject if none found
→ MockTest.create

Option B: POST /mocktest/add
→ admin check
→ MockTest.create({ title, subject, topic, duration, price, questions })

Students → GET /mocktest → see the new test
```

## 23.10 Data consistency across a full journey

| Event | Documents touched |
|---|---|
| Signup | User. |
| Open note | User.readNotes. |
| Test submit | Result; User.recentActivity; User.performanceStats.testsTaken. |
| Purchase | Purchase. |
| Certificate | Certificate; User.certificates. |
| Admin upload | Notes/PYQ/MockTest; GridFS or uploads. |

There are no multi-document transactions. A failure between “Result.create” and “addActivity” leaves a result that is not in the activity feed. A failure between certificate creation and the user link leaves a certificate that `/mine` will not show,

because `/mine` populates the user array.

CHAPTER 24

# Security and Code Quality Audit

> The application has a sound core—bcrypt password hashing, signed JWTs, server-side scoring, ownership checks on results and certificates, and escaped note rendering. The main weaknesses are inconsistent admin authorization, an unverified manual payment path, and missing operational hardening.

## 24.1 Risk summary

| Severity | Finding | Impact |
|---|---|---|
| **High** | Manual UPI accepts any UTR. | Any student can unlock a paid test without paying. |
| **High** | Admin stats, students and course creation are public. | Student emails are exposed; anyone can create courses. |
| **High** | Note/PYQ upload and delete require only a login. | Any student can replace or delete study material. |
| **High** | Registration does not enforce the strong-password helper. | Direct API calls can create weak-password accounts. |
| **Medium** | Purchase access ignores payment status. | A pending row would unlock a test. |
| **Medium** | Razorpay amount is not re-verified. | A mismatched captured amount is not detected. |
| **Medium** | No rate limiting on auth, AI, TTS or uploads. | Brute force, quota abuse, memory/disk exhaustion. |
| **Medium** | JWT in localStorage; no revocation. | XSS steals a 7-day token; logout is client-only. |
| **Medium** | Open CORS and no security headers. | Broad cross-origin access; weaker browser defenses. |
| **Low** | Unbounded upload size and no MIME allow-list. | Large files can exhaust memory or disk. |
| **Low** | Math.random for OTP. | Predictable OTPs if the PRNG is observed. |
| **Low** | Stale analytics values. | Misleading rank/hours/average. |
| **Low** | No request logging or health endpoint. | Harder incident response and monitoring. |

## 24.2 Authentication and authorization

### What is correct

- Passwords are hashed with bcrypt, cost 10, and the hash is `select:false`.
- JWTs are signed with `JWT_SECRET` and expire after seven days.
- Login returns the same “Invalid email or password” message for both wrong email and wrong password.
- Password reset stores an OTP hash, not the OTP, with a 10-minute expiry, 60-second resend cooldown and five-attempt limit.
- Reset tokens are purpose-scoped, single-use and stored as hashes.
- Results and certificates are queried with the current user id.

### What to fix

- Call `isStrongPassword` in registration. It currently exists but is unused there.
- Add a reusable `adminOnly` middleware and apply it to every admin route.
- Do not trust `isAdmin` from localStorage; the backend role check is the only authority.
- Consider short-lived access tokens plus refresh tokens, or HttpOnly secure cookies.
- Add a token version or denylist if immediate logout/revocation is required.
- Normalize emails to lowercase before storing and querying to avoid case-variant duplicates.

## 24.3 Admin route matrix

| Route | Protection | Verdict |
|---|---|---|
| `GET /admin/purchases` | protect + isAdmin | Correct. |
| `POST /admin/purchase/manual` | protect + isAdmin | Correct, but high impact. |
| `POST /admin/purchase/confirm` | protect + isAdmin | Correct. |

| Route | Protection | Verdict |
|---|---|---|
| `GET /admin/stats` | none | **Public.** |
| `GET /admin/courses` | none | **Public.** |
| `POST /admin/courses` | none | **Public write.** |
| `GET /admin/students` | none | **Public PII.** |
| `POST /notes/upload` | protect only | **Any user.** |
| `DELETE /notes/:id` | protect only | **Any user.** |
| `POST /pyq/upload` | protect only | **Any user.** |
| `DELETE /pyq/:id` | protect only | **Any user.** |
| `POST /mocktest/add` | protect + isAdmin | Correct. |
| `POST /mocktest/upload-pdf` | protect + isAdmin | Correct. |
| `POST /payment/settings/admin` | protect + isAdmin | Correct. |

## 24.4 Payment security

```
// current access rule
const purchase = await Purchase.findOne({ userId, testId });
return Boolean(purchase);
```

This ignores `status`. Tighten it:

```
const purchase = await Purchase.findOne({
  userId,
  testId,
  status: { $in: ["paid", "manual", "simulated"] },
});
```

- Manual UTR is self-reported. Require admin approval or a real provider webhook.
- After signature verification, fetch the Razorpay payment and compare amount, currency and status.
- Store the provider order amount and verify it equals the current test price.
- Do not expose a "simulate" path in production builds.

## 24.5 Input, uploads and files

- Multer uses memory storage for notes/PYQ with no file size limit. A large upload can exhaust RAM.
- There is no MIME or extension allow-list. Validate PDF magic bytes and cap at a sensible size (for example 20–50 MB).
- The QR upload accepts any extension and replaces the old file by path; validate image types and size.
- PDF parsing runs in-process. A malicious or malformed PDF could crash the worker; isolate parsing in a child process or queue.
- Mock-test PDFs remain in `uploads` with server-generated names but are not cleaned up when a test is deleted.

## 24.6 Data exposure

- `/api/admin/students` returns names, emails and student ids without a token.
- `/api/admin/stats` returns aggregate enrollment trends publicly.
- AI error responses include upstream `details`, which may reveal provider internals.
- The admin login stores the whole login response in localStorage, including `isAdmin` and token.
- No secret values are committed in the frontend; the Groq, Razorpay secret, Mongo URI and SMTP credentials stay server-side.

## 24.7 XSS and rendering

`renderPlainText` calls `escapeHtml` on every text fragment before inserting tags. The only way to inject markup would be to change that code to concatenate unescaped note content. The note body is therefore not an XSS sink today.

```
escapeHtml(str)
  .replace(/&/g, "&amp;")
  .replace(/</g, "&lt;")
  .replace(/>/g, "&gt;")
  .replace(/"/g, "&quot;")
  .replace(/'/g, "&#39;");
```

The code-preview iframe uses `sandbox="allow-scripts"` without `allow-same-origin`, so user HTML cannot access the parent origin. This is the correct choice.

## 24.8 Operational hardening checklist

| Control | Current state | Recommendation |
|---|---|---|
| Helmet / headers | Absent | Add CSP, HSTS, nosniff, frame-ancestors. |
| CORS | Allow all origins | Restrict to the deployed frontend origin. |
| Rate limiting | Absent | Limit login, OTP, AI, TTS and uploads. |
| TLS | Local HTTP | Terminate HTTPS at a reverse proxy. |
| Secrets | .env, not committed | Keep out of git; rotate on exposure. |
| Logging | Console only | Structured logs with request ids. |
| Backups | Not implemented | Atlas snapshots + uploads/GridFS backup. |
| Health check | No `/health` | Add liveness/readiness endpoints. |

## 24.9 Code quality observations

- `AdminDashboard.jsx` exceeds 1,000 lines; split by section.
- Auth headers are duplicated in every page; centralise an Axios instance with interceptors.
- Two chatbot files duplicate the same logic and hardcode `localhost`.
- Activity scores are stored as strings, forcing repeated parsing in analytics.
- Course/subject are free-text strings rather than references, so aliases and matching logic spread into multiple files.
- The same domain allow-list is copied in frontend and backend.
- There is no automated test suite; only a sample PDF exists under `backend/tests`.
- Some source files contain mojibake characters in strings and comments from encoding issues.

## 24.10 Prioritised remediation plan

1. Lock down the four public/PII admin endpoints and note/PYQ mutations with an admin guard.
2. Disable or gate the manual UTR flow; require verified payment.
3. Enforce strong passwords on the server and add auth/OTP rate limits.
4. Add upload size and type validation; clean up orphaned files.
5. Require a purchase status in `hasAccess`; re-verify Razorpay amount.
6. Add Helmet, restricted CORS, structured logging and health checks.
7. Move tokens to HttpOnly cookies or add refresh/revocation.
8. Refactor the admin dashboard and shared API client.

> **BALANCED VIVA ANSWER**
> "The authentication and scoring core is solid: bcrypt, signed JWTs, server-side scoring and ownership checks. The weaknesses are authorization consistency and payment verification. I can name the exact routes that need an admin guard and the exact predicate I would change."

CHAPTER 25

# Debugging Playbook

A practical sequence for starting the project, isolating failures, and verifying each layer from MongoDB to the browser.

## 25.1 Start order

```
# terminal 1
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\backend"
npm.cmd run dev

# terminal 2
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\frontend"
npm.cmd run dev
```

Open the Vite URL (normally `http://localhost:5173`). The backend listens on `5000` unless `PORT` is set. Start the backend from the `backend` directory because some code uses `process.cwd()` and relative `uploads/` paths.

**POWERSHELL NPM ISSUE**

If `npm` fails with a script-execution policy error, use `npm.cmd`. If `curl` behaves like `Invoke-WebRequest`, use `curl.exe` for real curl semantics.

## 25.2 Environment checklist

```
backend/.env
PORT=5000
JWT_SECRET=<long random string>
MONGO_URI=<MongoDB Atlas connection string>
GROQ_API_KEY=<optional for chat>

# optional
RAZORPAY_KEY_ID=...
RAZORPAY_KEY_SECRET=...
SMTP_HOST=...
SMTP_PORT=587
SMTP_USER=...
SMTP_PASS=...
SMTP_FROM=...
```

Do not print these values in screenshots, logs or the guide. If `JWT_SECRET` is missing, login token generation fails. If `MONGO_URI` is missing, the server exits immediately.

## 25.3 MongoDB connection failures

| Symptom | Likely cause / fix |
|---|---|
| `MONGO_URI is not defined` | `.env` missing or server started outside `backend`. |
| `MongooseServerSelectionError` | Atlas IP allow-list, wrong host, DNS, or network. |
| Times out after 5 seconds | The app deliberately uses a 5s selection timeout. Check Atlas access. |
| Authentication failed | Wrong database user/password or unescaped characters in URI. |
| Works locally, fails on another machine | Add that public IP to Atlas Network Access. |

The server calls `process.exit(1)` on database failure, so nodemon will wait for a file change unless it restarts automatically. Fix the environment, then save a file or restart.

## 25.4 Port already in use

```
Get-NetTCPConnection -LocalPort 5000 -State Listen |
  Select-Object LocalAddress, LocalPort, OwningProcess

Get-Process -Id <pid>
```

Stop the old Node process, or change `PORT` and update the Vite proxy. The same technique works for `5173`. Never run two backend copies against the same database without understanding the consequences.

## 25.5 Frontend cannot reach the API

Symptoms: login spins forever, "Cannot connect to server", 404 on `/api/...`.

1. Confirm the backend is listening: `Test-NetConnection localhost -Port 5000`.
2. Confirm Vite is running and the proxy is configured for `/api` and `/uploads`.
3. Check whether the page hardcodes `http://localhost:5000`; the chatbot does.
4. In production, confirm Express serves `frontend/dist` and the SPA fallback exists.
5. Check CORS only if the frontend calls the backend on a different origin without a proxy.

## 25.6 Authentication failures

| Symptom | Check |
|---|---|
| 401 "no token" | `localStorage.getItem("token")`; request header format must be `Bearer <token>`. |
| 401 "token failed" | Expired token, different `JWT_SECRET`, or malformed value. Log in again. |
| Login works, navbar still shows Login | Navbar reads `token`; a full reload may be needed because it has no auth context. |
| Admin link missing after student login | Normal login stores only the token, not `isAdmin`. Use `/admin-login` for admin UI. |
| Admin route redirects to login | `isAdmin` key missing/false; log in through the admin portal. |

To inspect a token payload, paste it into a JWT debugger or decode the middle segment. Never paste production tokens into third-party sites; decode locally instead.

## 25.7 Route ordering bugs

If `/api/mocktest/results/mine` returns a cast error or "test not found", check declaration order. Dynamic `/:id` must come after specific paths. The same rule applies to `/result/:resultId` and any future static subroutes.

## 25.8 Notes and PDF parser issues

- **Note has PDF but no text:** `pdf-parse` failed or the PDF is scanned images. The route returns the note with empty content; OCR is not implemented.
- **Headings look wrong:** the parser is heuristic. Check the source text around the line; short title-case lines may be misclassified.
- **PDF 404:** GridFS id missing or bucket name changed. Check `pdfs.files` in MongoDB.
- **Download works, viewer empty:** note has `fileId` but no extracted `content`; opening the note should trigger lazy extraction.
- **Huge note freezes the tab:** the viewer renders everything at once; split or paginate the content.

## 25.9 Mock-test upload issues

- **"No questions could be parsed":** the regex expects numbered questions and four options in a compatible layout. Try a text-based PDF, not a scan.
- **Wrong answer detected:** the parser matches "Answer: A" or an option string. Audit the parsed test before students use it.
- **Upload path error:** run from `backend` so `uploads/` resolves correctly, or create the folder.
- **Test list empty:** requests are protected. Confirm the token and that at least one test exists.

## 25.10 Payment issues

| Symptom | Check |
|---|---|
| UPI modal instead of Razorpay | Razorpay keys are absent; the backend returns `simulate: true`. |
| "Real Razorpay keys are configured" | `simulate-pay` refuses once keys exist. Use real checkout. |

| Symptom | Check |
|---|---|
| Payment succeeds but test locked | Check the purchase row status and `hasAccess`; confirm `userId` matches. |
| Signature verification fails | Wrong key secret, mismatched order/payment ids, or test/live key mix. |
| Duplicate purchase error | The unique `{userId, testId}` index; use upsert, not create. |

## 25.11 Certificate issues

```
GET /api/certificate/progress/BCA
```

Progress requires at least one note or test, every matching note in `readNotes`, and a result for every matching test. Common causes of “almost complete”:

- A note exists with a course spelling the regex does not match.
- The student downloaded a PDF instead of opening the viewer, so it was never marked read.
- A test subject uses a different spelling from the course pattern.
- The certificate was created but the user array was not linked; `/mine` will not show it.

## 25.12 AI and TTS issues

- **Chat fails:** missing/invalid `GROQ_API_KEY`, quota, or model unavailable. Check the server console.
- **Hindi audio silent:** Google TTS may be blocked or rate-limited; the frontend should fall back to browser speech.
- **Works on the server machine only:** the chatbot hardcodes `localhost:5000`; change to a relative or configured URL.
- **Long responses cut off:** TTS chunks text; very long text takes many sequential requests.

## 25.13 Build and deploy

```
cd "C:\Users\user\OneDrive\Desktop\project\Monitor Lab\frontend"
npm.cmd run lint
npm.cmd run build
npm.cmd run preview
```

The build writes `frontend/dist`. Start Express and open `http://localhost:5000` to test the production path. If deep links 404, the SPA fallback is missing or `dist` is in the wrong location.

## 25.14 Logs to watch

| Message | Meaning |
|---|---|
| `MongoDB Connected` | Database connection succeeded. |
| `Registration error` | Validation, duplicate email, or database failure. |
| `Login error` | Unexpected failure; bad credentials return 400 without this log. |
| `PDF text extraction failed` | Bad/scanned PDF; note may be saved with empty content. |
| `Error communicating with Groq API` | Provider/network/key problem. |
| `SMTP unavailable. OTP for ...` | Development fallback; never acceptable in production logs. |

## 25.15 Minimal smoke test

1. Start MongoDB connection and backend; confirm the port.
2. Sign up with a valid provider email and strong password.
3. Refresh; confirm the session persists.
4. Open a note; confirm it renders and is marked read.
5. Attempt a free test; confirm score, review and activity row.
6. Open analytics; confirm a trend appears.
7. Log in as admin; upload a note and a test.
8. Run frontend lint and build; serve `dist` from Express and deep-link to a route.

**DEBUGGING PRINCIPLE**

Test one layer at a time: database reachable → backend route returns → proxy forwards → component receives → UI renders. Guessing at the UI before confirming the API wastes the most time.

V

# Viva and Revision

The numbers worth memorising, short answers to the questions examiners actually ask, and a final file map, demo script and verification checklist.

CHAPTER 26

# Numbers and Details to Memorize

> Exact figures make a viva sound grounded. These are the numbers that come directly from the code, not estimates.

## 26.1 Counting the system

| Item | Value |
|---|---|
| Backend models | 10: User, MockTest, Notes, PYQ, Result, Purchase, Certificate, Setting, Course, Lecture. |
| Route modules | 8: auth, admin, mocktest, notes, pyq, ai, payment, certificate. |
| API endpoints | 44 (counting GET + PUT `/profile` separately). |
| Frontend routes | 28. |
| Admin dashboard sections | 7. |
| GridFS buckets | 2: `pdfs`, `pyqs`. |
| Purchase statuses | 4: paid, manual, simulated, pending. |
| Certificate courses | 3: BCA, BBA, BIOTECH. |
| PYQ universities in the UI | 6. |
| Compiler languages | 6 (including HTML/CSS preview). |

## 26.2 Authentication numbers

| Item | Value |
|---|---|
| JWT lifetime | 7 days. |
| bcrypt salt rounds | 10. |
| OTP length | 6 digits. |
| OTP lifetime | 10 minutes. |
| OTP resend cooldown | 60 seconds. |
| OTP maximum attempts | 5. |
| Signup minimum password | 8 characters + letter + number + special character. |
| Login minimum length | 6 characters (client check only). |
| Reset token lifetime | 10 minutes. |

## 26.3 Mock-test numbers

| Item | Value |
|---|---|
| Correct answer | +4 marks. |
| Wrong answer | −1 mark. |
| Unattempted | 0 marks. |
| Maximum score | 4 × number of questions. |
| Default duration | 60 minutes. |
| Timer conversion | `duration × 60` seconds. |
| Razorpay unit | Paise; price × 100. |
| Default price | ₹0 (free). |
| Questions per PDF heuristic | Up to 4 options per question. |

## 26.4 Certificate numbers

| Item | Value |
|---|---|
| Certificate id format | `ML-YYYYMMDD-XXXXXXXX`. |
| Random suffix | 4 bytes → 8 uppercase hex characters. |
| Completion condition | All notes read + all tests attempted + at least one item. |

| Item | Value |
|---|---|
| Score requirement | None. |
| Certificate snapshot | Name, college, phone, email, student id, course, issue date. |

## 26.5 Content numbers

| Item | Value |
|---|---|
| AI PDF context cap | 8,000 characters. |
| TTS chunk size | About 180 characters, split at word boundaries. |
| Reading speed estimate | 200 words per minute. |
| Notes list payload | Excludes `content` via projection. |
| PDF TOC scan limit | First 100 lines. |
| Activity trend window | Last 10 activities. |
| Monthly chart window | Last 6 months. |
| Admin registration trend | 6 months. |

## 26.6 Exact strings and shapes

```
JWT header: Authorization: Bearer <token>
JWT payload: { id }
JWT expiry: 7d
Mock-test answer payload: { "0": 2, "1": 0 }
Manual purchase orderId: "manual", paymentId: "manual"
Simulated orderId prefix: "sim_"
Certificate id example: ML-20261025-A1B2C3D4
GridFS note bucket: pdfs
GridFS PYQ bucket: pyqs
Mongo purchase index: { userId: 1, testId: 1 } unique
```

## 26.7 Correct vs incorrect statements

| Say this | Not this |
|---|---|
| “Scores are calculated on the server.” | “The browser sends the score.” |
| “Correct answers are hidden in the test payload.” | “The client knows all answers.” |
| “The purchase table is unique per student/test.” | “A student can have many purchase rows.” |
| “Certificates require completion, not a passing score.” | “A certificate requires 40%.” |
| “Notes are stored in GridFS and text is extracted.” | “PDFs are rendered natively in React.” |
| “AdminRoute is a UX guard; the API enforces roles.” | “Hiding the admin button secures the API.” |
| “Razorpay verification uses an HMAC signature.” | “The frontend confirms the payment.” |
| “Manual UTR flow is a demo, not verification.” | “UTR proves the payment.” |
| “AI and TTS are proxied by the backend.” | “The Groq key is in the frontend.” |
| “The HTML preview iframe is sandboxed.” | “User HTML runs with full app access.” |
| “Three admin data routes are currently public.” | “Every admin route is protected.” |
| “Activity is unbounded.” | “Activity is capped with $slice.” |

## 26.8 One-line definitions

| Term | Definition |
|---|---|
| JWT | A signed token carrying a user id, verified on each protected request. |
| bcrypt | A deliberately slow password hash with a per-password salt. |
| GridFS | MongoDB’s chunked file storage for files above the document limit. |
| Mongoose | Schema/model layer over the MongoDB driver. |
| Vite | Frontend dev server and production bundler. |

| Term | Definition |
| --- | --- |
| SPA fallback | Express returns `index.html` for unknown paths so React Router can handle them. |
| Upsert | Update if present, insert if absent. |
| Idempotent | Repeating the operation does not create a duplicate effect. |
| HMAC | Keyed hash used to verify Razorpay's signature. |
| Sandbox iframe | An isolated frame with restricted capabilities. |
| Stale closure | A callback that captured old state because its effect did not re-run. |
| Denormalisation | Storing summary/snapshot data to avoid joins or preserve history. |

CHAPTER 27

# Rapid-Fire “Why” Answers

> Short, defensible answers to the questions examiners actually ask. Each answer names a trade-off rather than pretending the design is perfect.

## Why did you choose the MERN stack?

One language across the stack, JSON end to end, and fast iteration. MongoDB suits document-shaped content like tests with embedded questions, and React fits a highly interactive exam UI.

## Why MongoDB instead of MySQL?

The test document naturally contains an array of questions and options, so embedding avoids a join-heavy schema. The trade-off is weaker relational constraints, which is why some integrity checks live in application code.

## Why JWT instead of sessions?

Stateless authentication works well for an SPA and a separate API; the server does not need a session store. The trade-off is revocation: a stolen token remains valid until expiry.

## Why store the JWT in localStorage?

It is simple and works with the Vite proxy and Bearer headers. It is also readable by JavaScript, so an XSS would steal it. HttpOnly cookies are the safer production choice.

## Why bcrypt?

It is intentionally slow and salted, making offline brute force expensive. Ten rounds balance security and login latency.

## Why hide the password field?

`select:false` keeps the hash out of normal queries. Login explicitly requests `+password` only when it needs to compare.

## Why server-side scoring?

The browser is untrusted. The server owns the correct answers, computes +4/−1, and stores the result. The client cannot inflate a score.

## Why +4 and −1?

It mirrors common competitive-exam marking: reward correct knowledge and penalise guessing, while leaving unattempted questions neutral.

## Why hide correct answers in the list and test payload?

A student could inspect network responses. Projecting out `questions.correctAnswer` prevents casual cheating; the review endpoint reveals them only after submission and only to the owner.

## Why a separate Purchase collection?

Access is a many-to-many relationship between users and tests. A join collection with a unique pair supports free, paid, manual and simulated states cleanly.

## Why the unique compound index on Purchase?

It makes access idempotent: repeated orders or verification cannot create duplicate rows. Upsert relies on it.

## Why does hasAccess ignore status?

It is a simplification, not a good final design. It should require a paid/manual/simulated status so a pending row cannot unlock content.

### Why is there a manual UPI path?

It lets the project demonstrate checkout when Razorpay keys are unavailable. It is explicitly labelled simulated and is not real payment verification.

### Why is Razorpay amount multiplied by 100?

Razorpay's API expects the smallest currency unit, paise. ₹99 becomes 9900.

### Why verify the Razorpay signature?

The HMAC proves the order/payment pair was signed with the server's secret. Without it, a client could claim a fake payment.

### Why use GridFS for PDFs?

PDFs can exceed the 16 MB document limit and should stream. GridFS chunks them and stores metadata; the document keeps only the file id.

### Why extract text from PDFs?

It enables a responsive web reader, searchable headings and reading progress. The PDF remains available for download.

### Why is the markdown parser heuristic?

Text extraction loses font and layout information. The parser infers headings from numbering, uppercase lines and PDF table-of-contents patterns. It is best-effort, not a full document model.

### Why escape note text before using dangerouslySetInnerHTML?

The parser generates HTML, so React's normal escaping is bypassed. Escaping every text fragment before adding tags prevents stored XSS.

### Why store a certificate snapshot?

A certificate is a historical record. If a student later changes college or name, the issued certificate should not silently change.

### Why isn't a score required for a certificate?

The product defines completion as consuming all material and attempting all tests. A score threshold would be a policy change, not a technical one.

### Why normalise course aliases?

URLs and content use variants like `bio`, `biotech` and `biotechnology`. Normalisation keeps progress and certificate lookup consistent.

### Why is the same test allowed multiple times?

Students benefit from retakes, and the Result collection preserves each attempt. Analytics treats each attempt as activity; a unique result index would remove that ability.

### Why is activity stored on the user?

It keeps the profile and analytics fast with one document read. The cost is an unbounded array and duplicated summary data; a separate Activity collection would scale better.

### Why don't rank and hours update?

The UI expects them, but the backend only increments `testsTaken`. They are placeholders. A real implementation would aggregate results or maintain a scheduled leaderboard.

### Why React?

The exam screen has many interdependent pieces of state—answers, palette, timer, results. Component state and declarative rendering make that manageable.

### Why Vite?

Fast dev startup, hot module replacement and a simple proxy for `/api` and `/uploads`.

### Why React Router?

It gives clean URLs, nested layouts, route params and client-side navigation without full reloads.

### Why are ProtectedRoute and AdminRoute not security?

They run in the browser and read editable localStorage. They improve navigation; the server middleware and role checks are the actual boundary.

### Why does the admin dashboard still work with public APIs?

Because some admin read routes lack guards. The UI sends a token for many calls, but the server does not require one for stats, students or course creation.

### Why use a Vite proxy?

It makes `/api` same-origin during development, avoiding CORS and keeping URLs identical to production.

### Why does the chatbot hardcode localhost?

It was written independently of the shared API pattern. It is a deployment bug, not a design choice; replace it with a relative URL.

### Why proxy AI and TTS through the backend?

The Groq key must stay server-side, and Google TTS blocks direct browser calls. The backend also normalises responses.

### Why is the AI endpoint unauthenticated?

It was built as an open helper. That exposes quota and enables abuse; it should be authenticated and rate-limited.

### Why sandbox the code preview?

User HTML can contain scripts. Omitting `allow-same-origin` isolates it from the app's storage and DOM.

### Why send code to Wandbox?

Running arbitrary languages securely requires containers or VMs. Wandbox provides that externally; the trade-off is privacy, latency and availability.

### Why is the timeout submit potentially wrong?

The timer effect captures an old `handleSubmit` closure. Without a ref or updated dependency, it can submit stale answers. This is a React stale-closure bug.

### Why show a zero result when submit fails?

It is a bug, not intended behaviour. The catch block fabricates zeros instead of surfacing an error and allowing retry.

### Why no database transactions?

The flows are mostly single-document and the project prioritised simplicity. Multi-step operations like result + activity + certificate are therefore not atomic.

### Why is there no automated test suite?

The project is a prototype and verification was manual. The highest-value tests would cover auth, payment verification, scoring and certificate completion.

### What would you fix first?

Authorization consistency: protect the public admin endpoints and note/PYQ mutations, then remove or gate the unverified UTR flow.

### What is the strongest part of the project?

The end-to-end learning loop: content ingestion, server-scored assessment, analytics and automatic certificate issuance, all tied to one account.

### What is the weakest part?

Security hardening around admin operations and payments, plus the lack of automated tests.

### How would you scale it?

Move files to object storage, add Redis for sessions/rate limits, queue PDF parsing, split the admin SPA, add indexes for results, and introduce a proper payment webhook.

### Why keep the PDF after extracting text?

Students may want the original layout, and extraction can be imperfect. The PDF is the source of truth; the text is a reading convenience.

### Why does the certificate route create the certificate on a GET?

It makes issuance automatic when the student checks progress. A stricter REST design would use a POST or a background job, but the side effect is idempotent because an existing certificate is reused.

### Why is the app name inconsistent?

The UI says “Mentor Lab”, the frontend package is `drona-gyan`, and some footers say “Drona Gyan”. It reflects the project’s evolution. The backend package is `monitor-lab-backend`.

CHAPTER 28

# File Map and Final Checklist

> A complete orientation to the repository, the commands you need, the code paths worth opening during a viva, and the checks to run before calling the project finished.

## 28.1 Repository root

```
Monitor Lab/
  backend/                Express API, models, uploads
  frontend/               Vite + React SPA
  Mentor-Lab-Viva-QA.pdf  existing viva deliverable
  Mentor-Lab-Viva-QA.docx existing viva deliverable
  Mentor-Lab-Viva-QA.doc  existing viva deliverable
```

## 28.2 Backend map

| Path | Responsibility |
|---|---|
| `src/index.js` | Express bootstrap, CORS, JSON, route mounts, static uploads, SPA fallback, listen. |
| `src/db.js` | Mongo connection with a 5s timeout and Atlas whitelist hint. |
| `routes/authRoutes.js` | Signup, login, password reset, profile, activity. |
| `controllers/authController.js` | Validation, bcrypt, JWT issue, OTP/reset logic, profile update. |
| `middleware/authMiddleware.js` | `protect`: Bearer token → verified user. |
| `utils/generateToken.js` | JWT with `{ id }` and 7-day expiry. |
| `utils/mailer.js` | SMTP transport and OTP email template. |
| `routes/mockTestRoutes.js` | Upload/parse, add, list, access, order, verify, manual pay, submit, results, review. |
| `routes/notesRoutes.js` | GridFS note upload, list/filter, read, lazy text extraction, PDF stream, delete. |
| `routes/pyqRoutes.js` | GridFS PYQ upload, filtered list, PDF stream, delete. |
| `routes/certificateRoutes.js` | Read-note marker, progress calculation, issuance, list, owner read. |
| `routes/paymentRoutes.js` | Public/admin settings, QR image disk storage. |
| `routes/adminRoutes.js` | Purchases, manual unlock/confirm, stats, courses, students. |
| `routes/aiRoutes.js` | Chat, note explanation, TTS proxy. |
| `controllers/aiController.js` | Groq calls, language prompt, paragraph formatting, PDF context. |
| `controllers/ttsController.js` | Google Translate TTS fetch and audio response. |
| `model/user.js` | Profile, auth fields, activity, read notes, certificates. |
| `model/mocktest.js` | Test metadata with embedded questions. |
| `model/result.js` | Per-attempt score and answer review. |
| `model/purchase.js` | Access rows with unique user/test index. |
| `model/certificate.js` | Issued certificate snapshot. |
| `model/notes.js`, `pyq.js` | PDF metadata and GridFS references. |
| `model/setting.js`, `course.js`, `lecture.js` | Payment settings and lightweight catalogue models. |
| `uploads/` | QR image and mock-test source PDFs. |
| `createAdmin.js` | One-off admin seeding script; contains credentials and must be sanitized. |
| `seedMockTest.js` | Sample test seeding helper. |

## 28.3 Frontend map

| Path | Responsibility |
|---|---|
| `src/main.jsx` | React root, StrictMode, ThemeProvider. |
| `src/App.jsx` | Navbar, 28 routes, chatbot widget, theme class. |

| Path | Responsibility |
|---|---|
| `src/components/ProtectedRoute.jsx` | Token-presence navigation guard. |
| `src/components/AdminRoute.jsx` | Token + local `isAdmin` guard. |
| `src/components/Chatbot.jsx` | Floating chat, language selector, TTS playback. |
| `src/components/CodeCompiler.jsx` | Wandbox runner and sandboxed HTML preview. |
| `src/context/ThemeContext.jsx` | Dark/light state provider. |
| `src/pages/home.jsx` | Landing page, course cards, QR registration. |
| `src/pages/login.jsx`, `signUp.jsx` | Student auth screens. |
| `src/pages/AdminLogin.jsx` | Admin portal using the shared login endpoint. |
| `src/pages/ForgotPassword.jsx` | Email → OTP → new password wizard. |
| `src/pages/MockTest.jsx` | Select, attempt, pay, auto/manual submit, result. |
| `src/pages/TestReview.jsx` | Per-question answer review. |
| `src/pages/StudyNotes.jsx` | Note catalogue and filters. |
| `src/pages/NoteViewer.jsx` | Parsed note, TOC, progress, read marker. |
| `src/utils/markdown.js` | Text → headings/paragraphs/lists with escaping. |
| `src/pages/PYQ.jsx` | University carousel and paper lookup. |
| `src/pages/RecordedLectures.jsx` | Static video and playlist. |
| `src/pages/PerformanceAnalytics.jsx` | Derived accuracy, trends, subjects, streak. |
| `src/pages/Profile.jsx` | Profile, college editing, certificates, activity. |
| `src/pages/MyCertificates.jsx` | Certificate grid/empty state. |
| `src/pages/CertificatePage.jsx` | Printable certificate. |
| `src/pages/AdminDashboard.jsx` | Seven-section admin panel. |
| `src/pages/ChatbotPage.jsx` | Full-screen chat route. |
| `src/pages/bca.jsx`, `bba.jsx`, `biotech*.jsx` | Course and category pages. |
| `src/pages/roadmapbca.jsx`, `bbaroadmap.jsx` | Roadmap pages. |
| `src/css_files/` | Page-specific CSS and shared variables. |
| `vite.config.js` | React/Babel plugins and `/api`, `/uploads` proxy. |
| `firebase/firebase.js` | Firebase/Auth placeholder, not wired into login. |

## 28.4 Command cheat sheet

```
# install (once per folder)
cd backend
npm.cmd install
cd ..\frontend
npm.cmd install

# development
cd backend
npm.cmd run dev
# second terminal
cd frontend
npm.cmd run dev

# production build
cd frontend
npm.cmd run lint
npm.cmd run build

# serve built app from Express
cd backend
npm.cmd start
```

```
# Windows port diagnosis
Get-NetTCPConnection -LocalPort 5000 -State Listen
Test-NetConnection localhost -Port 5000

# health of backend (no dedicated health route; SPA root)
curl.exe -i http://localhost:5000/

# login example
curl.exe -X POST http://localhost:5000/api/auth/login -H "Content-Type: application/json" -d "
{\"email\":\"student@example.com\",\"password\":\"Secret123!\"}"
```

## 28.5 Runtime configuration

| Variable | Required | Effect if missing |
|---|---|---|
| `MONGO_URI` | Yes | Server exits. |
| `JWT_SECRET` | Yes | Token signing/verification fails. |
| `PORT` | No | Defaults to 5000. |
| `GROQ_API_KEY` | For AI | Chat/explain return 500. |
| `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET` | For real payments | Manual simulation mode. |
| `SMTP_HOST`, `SMTP_USER`, `SMTP_PASS` | For OTP email | OTP logged; not emailed. |

## 28.6 Demo script for a viva

1. **Architecture** — show the folder tree and explain the SPA → proxy → Express → Mongo/GridFS path.
2. **Auth** — sign up, show the bcrypt hash in the database, log in, show the JWT and the profile call.
3. **Content** — open a note, show extracted text, TOC, progress and the read marker.
4. **Assessment** — start a free test, answer a few questions, submit, show the +4/−1 result and review.
5. **Integrity** — point out that correct answers are absent from the test payload and scoring happens on the server.
6. **Analytics and certificate** — show the activity row, trend, and progress endpoint; explain the completion rule.
7. **Admin** — upload a note or create a test, then show the database record and the student view.
8. **Audit** — name the public admin routes and the unverified UTR flow, then describe the exact fixes.

## 28.7 Final verification checklist

| Check | Pass condition |
|---|---|
| Backend installs and starts | `npm.cmd run dev` connects to Mongo and listens on 5000. |
| Frontend installs and starts | Vite serves the app; no console build errors. |
| Signup/login | Token persists; profile loads; invalid credentials rejected. |
| Password reset | OTP delivered or logged; reset token works once; old token fails. |
| Notes | List, read, PDF download, lazy extraction, delete. |
| PYQ | Upload, filter by university/course/subject, open PDF. |
| Mock tests | Free and paid access, scoring, result, review, owner isolation. |
| Certificates | Progress correct; certificate issued once; owner-only read. |
| Admin | All seven sections load; uploads and purchases work. |
| Analytics | Trend, accuracy and history reflect real attempts. |
| Lint/build | `npm.cmd run lint` and `npm.cmd run build` complete. |
| Production serve | Express serves `dist`; deep links fall back to `index.html`. |
| Secrets | No credentials in source, screenshots, logs or the guide. |
| Deliverables | New HTML/PDF/DOCX/DOC present; old viva files unchanged. |

## 28.8 Known limitations

- Manual UPI is not verified; there is no payment webhook.

- Several admin read/write routes are public or only login-protected.
- No rate limiting, security headers or centralized validation library.
- No automated tests; no CI pipeline.
- PDF parsing is heuristic and has no draft-review step.
- Analytics rank/hours/average are placeholders.
- Activity is unbounded and duplicated across documents.
- Chatbot/TTS URLs hardcode localhost.
- Recorded lectures are static; the Course/Lecture models are underused.
- No notification, email verification or account deletion flow.

## 28.9 Recommended next steps

1. Add `adminOnly` middleware and apply it consistently.
2. Replace the simulated payment path with verified Razorpay webhooks or admin approval.
3. Add rate limits, upload limits, Helmet and restricted CORS.
4. Add integration tests for auth, scoring, payment and certificate completion.
5. Move file storage to S3-compatible object storage for scale.
6. Introduce a real activity/leaderboard collection and scheduled aggregates.
7. Split the admin dashboard and create a shared API client.
8. Add content preview/edit before publishing parsed tests.

## 28.10 Final summary

Mentor Lab is a full-stack learning platform built around a complete student loop: discover a course, read structured notes, attempt server-scored mock tests, review mistakes, track progress, and earn a completion certificate. It combines a React SPA with an Express/MongoDB API, GridFS file storage, JWT authentication, Razorpay integration with a demo fallback, and AI/TTS study helpers.

The engineering strengths are clear separation of frontend and backend, server-side assessment integrity, ownership checks on personal records, and pragmatic use of embedded documents for quiz content. The engineering risks are equally clear: inconsistent admin authorization, unverified manual payments, and missing operational hardening. A good viva answer should present both—what works, why it was chosen, and exactly what you would change next.

> **CLOSING NOTE**
> Know the request path, know the data model, know the scoring formula, and know the three security fixes. Those four things cover most of the project's technical story.